An Investigation into the Restricted Boltzmann for Modelling the 2D Ising Model

Steven Glass and Johann Matthee

Introduction

Traditionally, investigating the properties of the Ising model has solely relied on Monte Carlo simulation methods, such as the Metropolis single flip algorithm, or the Wolff cluster flip algorithm. However, we saw in previous investigations that these algorithms can often be extremely computationally intensive for larger lattice sizes. Additionally, they suffer from what is known as critical slowing down near the phase transition at $T = T_c$. At this temperature, it can be shown that the correlation length of the system diverges, which results in extremely long equilibration times for temperatures near the critical temperature.

In this investigation, we will explore the possibility of implementing a machine learning algorithm known as the Restricted Boltzmann Machine (RBM) to model the behaviour of the Ising model. In this report, we are going to formally motivate the use of the RBM for modelling the Ising model from first principles. We will explore how unsupervised machine learning models the underlying distribution of data, and uses this "learned" distribution to generate new observations. Additionally, we will explore the genesis of the RBM from it's non-stochastic predecessor the Hopfield Network, and delve deep into interpreting and understanding the final trained RBM. Finally, we will conclude by implementing the RBM and test how well it is able to model the behaviour of the 2D Ising model.

Part 1: An Introduction through the Hopfield Network

The goal of this section is to introduce the reader to some of the fundamental concepts and innovations underlying the invention of the RBM. These concepts are not new, and form a large part of the foundation of modern day machine learning. However, to limit the scope of the discussion, these concepts will only be explored as far as they pertain to the current focus of Restricted Boltzmann Machines. With that being said, the proceeding section will cover the concepts by introducing them through the Hopfield Network, as this establishes a clear and easily explainable example. Most of Part 1 is based on [1] and [2].

Firstly a brief definition of the Hopfield Network will be given, thereafter everything of relevance will be systematically unpacked and explained.

The Hopfield Network is a set of fully connected neurons with each connection being bidirectional and symmetric. Certain inputs can be fixed in the network with Hebbian Learning which represent local minima in an energy space. Hopfield Networks can reconstruct the fixed inputs by sequentially applying an update rule to the different neurons to minimize the energy of the system until it reaches a local minima.

From the definition of the Hopfield Network there are 3 main subsections that merit further clarification and explanation. First, the mathematical objects necessary for constructing the Hopfield Network will be defined. Thereafter, it will be explained precisely what the Hopfield Network is trying to achieve and how it goes about doing so. Finally, an explanation will be given of how the Hopfield network learns inputs and reconstructs them.

1.1 Neurons and Connections

The first of these subsections will explain the general architecture of the Hopfield Network.

The Hopfield Network is constructed out of neurons which in this context are similar to spins states. That is, for the Hopfield Network of size N, x_i is the ith neuron of the Hopfield Network where $x_i \in \{-1, +1\}, i = 1, 2, ..., N$. The size of the network is dependent on the size of the system that is simulated. In the context of the Ising model with a lattice of size $L \times L$, a Hopfield Network requires L^2 neurons to be able to represent a possible spin configuration.

Hopfield Networks are fully connected graphs, meaning that every node, in this case neuron, is connected to every other node in the network. These connections are bidirectional, going both ways. Furthermore, the weights of the network are symmetric. That is, given a connection between x_i and x_j with weight w_{ij} and the connection between x_j and x_i with weight w_{ji}, then $w_{ij} = w_{ji}$. The weights of the Hopfield Network represent how strong two neurons are coupled. A very basic understanding of the strength of neuron coupling, as it pertains to the Ising model, is how strong the state of one spin is correlated with the state of another spin. The relevance of the coupling strength will be explained in the proceeding subsection.

1.2 Update Rules and Optimization

1.2.1 Update Rule

In the following subsection the update rule for the Hopfield Network will be explained, as well as what type of problem the Hopfield Network is trying to solve.

An update rule is a mathematical definition of how to change certain aspects of a system based on some predefined conditions. In the case of the Hopfield Network, the update rule specifies under which conditions to change the state of a neuron. The following components of the network are subject to a specified update rule: The weights between the neurons and the the states of the each neuron x_i and all the neurons connected to it $x_j \in N, \forall j \neq i$.

The connection between two neurons can be classified as excitatory or inhibitory. Excitatory connections are when the weight $w_{ij} > 0$ between a pair of two neurons x_i and x_j. An inhibitory connection is when $w_{ij} < 0$ between a pair of neurons. In the context of the Ising model, two states are excitatory if there is a positive correlation between their spins aligning, and inhibitory if there is a negative correlation indicating spin misalignment.

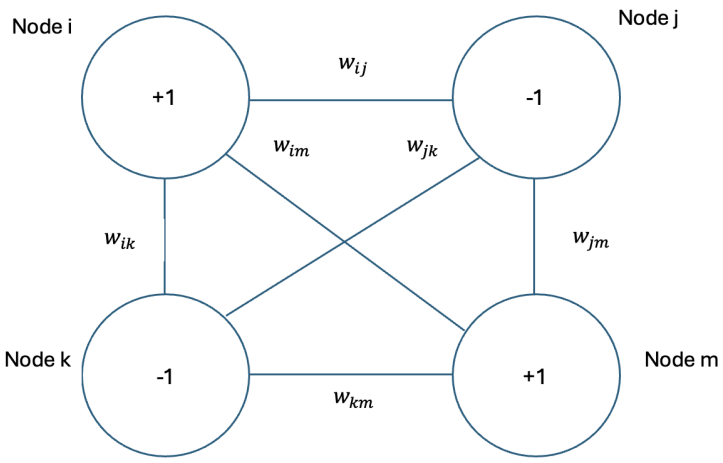

Figure 1: A simplified example of nodes and their connections within the Hopfield Network

If x_i and x_j are aligned their product $x_i x_j > 0$. To reinforce this alignment, the product $w_{ij} x_i x_j$ should ideally be greater than zero, since when the spins align, both the weight w_{ij} and the product $x_i x_j$ are positive, whilst if they misalign their product and weight are both negative.

An additional variable h_i is defined to make the update rule more compact. Keeping i fixed, h_i is defined as:

$$h_i = \sum_{j \neq i}^{edges} w_{ij} x_j$$

Here h_i represents the net input to neuron x_i from the rest of the network. If $w_{ij} > 0$ and x_i is misaligned with x_j then a negative value will be added to hi, and the same holds if x_i is aligned with x_j and $w_{ij} < 0$. Therefore the greater h_i the more x_i is in agreement with the rest of the network.

Thereafter, the update rule for the Hopfield Network can then be defined as:

$$x_i = \begin{cases} +1, & \text{if } h_i > 0 \\ -1, & \text{if } h_i < 0 \end{cases}$$

Intuitively this makes sense, since if h_i is a large negative value and $x_i = +1$, that would imply that x_i is extremely misaligned to spins it should be aligned with, and extremely aligned with spins it should not be aligned with; thus its spin should therefore be flipped. The same holds in the opposite case.

1.2.2 Optimisation Problem

The concept of an update rule is then extended further to all neurons, and forms the optimisation problem that the Hopfield Network tries to solve. In the case of Hopfield Networks the optimisation aims to maximise the following objective function:

$$\text{Goal: Maximise} \sum_{j \neq i}^{edges} w_{ij} x_i x_j$$

Intuitively, if h_i is a measure of how much x_i agrees with the rest of the network, it is understandable that the summation of how much each neuron agrees with the network would be a good measure of much the network is in overall agreement. Furthermore, this maximisation problem is equivalent to:

$$\text{Goal: Minimise} - \sum_{j \neq i}^{edges} w_{ij}\, x_i x_j$$

which is the measure of the overall disagreement between the neurons.

When comparing the Hopfield Network's optimisation problem with the Hamiltonian of the Ising model it is clear that, at some temperatures, the Ising model also optimises its own energy function. Consider the Ising model's Hamiltonian:

$$H(\mathbf{s}) = -\sum_{\langle i,j \rangle} J\, s_i s_j$$

which only favours aligned spins (excitatory connections) since $J > 0$. It is observed that when the system has a lot of energy (at high T), there is little restriction or optimisation performed between the

different spins thus the system is not concerned with minimising its energy. When the system has little energy, at low T, spins start to align with neighbouring spins in an attempt to optimise the energy (minimize the total energy).

Following the convention of the Ising model, by minimising the energy at low temperatures, the minimisation problem of the Hopfield Network will also be defined as minimising the energy of the system:

$$\text{Energy} = -\sum_{j \neq i}^{edges} w_{ij} x_i x_j$$

With energy here representing the total disagreement between neurons in the network.

With the optimisation problem now clearly defined, the update rule is then sequentially applied to each neuron in the network, and the updating sweep is then performed multiple times over all the neurons each iteration bringing the network into more overall agreement.

1.3 Energy Landscapes, Hebbian Learning, and Reconstructing Input

From the previous subsections it is clear how Hopfield Networks are build and how they optimise the networks energy through the update rule. However, what is not clear is how the network learns and recalls information based on inputs. The subsequent subsection will explain how a Hopfield Network learns inputs and how it is able to reconstruct those inputs.

1.3.1 Energy Landscape

A crucial concept in all of machine learning is the idea of a parameter space, and in the case of Hopfield Network this is known as the energy landscape. In general, a parameter space is a multi-dimensional space in which each dimension corresponds to a model parameter, and each unique set of parameter values corresponds to a point within this space. Each point has a corresponding function value, and generally this is represented by a cost function value (such as the sum of squared residuals in linear regression), which quantifies how well the model is performing given it's current set of parameters. In the case of the Hopfield Network we replace the cost function with the energy of the system $\mathrm{E}(\mathbf{x}, \mathbf{W})$.

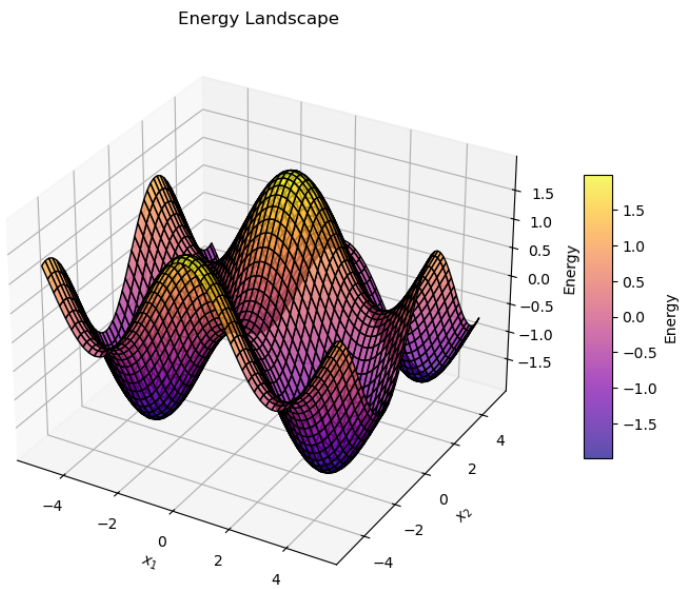

Figure 2: Energy Landscape Visualisation for 2 model parameters

The shape of the landscape is dictated by the weights of the Hopfield Network, and the each point on the landscape represents some possible combination of neuron states. More precisely for a fixed number of neurons and fixed weights, the energy landscape represents all possible combinations of neuron states, and their respective energy based on the weights of the network. If the weights changed the shape of the landscape would change as well. This can expressed more formally by noting that:

$$\mathbf{y} = \mathbf{W}^T\mathbf{x}$$

Where $\mathbf{W} \in \mathbb{R}^{N\times N}$ is the weight matrix, with $(i,j)^{th}$ entry being weight w_{ij} connecting neurons x_i and x_j. Similarly $\mathbf{x} \in \mathbb{R}^N$ is the location vector, where the ith entry represents neuron x_i in the network. In the equation $\mathbf{y}$ represents the energy vector, where each $y_i, i = 1, 2, ..., N$ represents the energy contributions of the state of neuron x_i to the the network. The energy at a given location on the energy landscape is the equal to the energy that each neuron contributes at that specific location $\mathbf{y}$, multiplied by the location vector $\mathbf{x}$. Thus the following equation holds:

$$\text{Energy} = \mathbf{y}^T\mathbf{x}$$

More compactly:

$$\text{Energy} = \mathbf{x}^T\mathbf{W}\mathbf{x}$$

1.3.2 Hebbian Learning

With the Energy Landscape explained, it still has to be shown how the network learns different inputs. The Hopfield Network learns and "remembers" different input by shaping and altering the energy landscape. The method by which the network does this is known as Hebbian Learning. To understand Hebbian Learning, first define some input that should be stored $\mathbf{v}$. Then set $w_{ij} = v_i v_j, \forall i, j$. By doing this the network creates a landscape that has an local minima at location $\mathbf{v}$. This is due to the fact that the energy function is at a minimum if $w_{ij} = v_i v_j$, because then all weights and neurons are in perfect agreement leading to the smallest possible energy for a given Hopfield Network. Hebbian learning can be formulated as:

$$\text{Energy}(\mathbf{v}) = -\sum_{j\neq i}^{edges} w_{ij}v_iv_j = -\sum_{j\neq i}^{edges} v_i^2 v_j^2$$

The current formulation only supports the storing of one input, $\mathbf{v}$, but it can easily be extended to more inputs. Given some set of inputs $I = \{\mathbf{v}_1, \mathbf{v}_2, ...\mathbf{v}_N\}$, the energy landscape can be changed to create a local minima for each $\mathbf{v}_n \in I$ by setting $w_{ij} = v_{ni}v_{nj}, \forall i, j$ and then summing all w_{ij} together.

$$\text{Energy}(I) = -\sum_{n=1}^{N}\sum_{j\neq i}^{edges} w_{ij}v_{ni}v_{nj} = -\sum_{n=1}^{N}\sum_{j\neq i}^{edges} v_{ni}^2 v_{nj}^2$$

This method is easy and efficient as long as there are not too many inputs to store and theses inputs are not close to one another in the energy landscape. If the before mentioned conditions are not met, then some local minima will start to overlap and this will lead to incorrect results.

1.3.3 Input Reconstruction

So far it has been shown what the energy landscape is, and how the Hopfield Network uses Hebbian Learning to change the shape of the landscape to store the location of inputs as local minima in the

energy landscape. The reason why it is useful that the network is able to do this, is due to the fact that now, all that must be done is minimise the energy of any given input, and the network will automatically update the different neurons until it reaches a local minima, which represents a learned input.

Let $\mathbf{z}$ be some random input situated somewhere on the energy landscape with energy α and let $\mathbf{v}$ be some learned input with energy β. The distance between $\mathbf{z}$ and $\mathbf{v}$ represents how similar $\mathbf{z}$ is to $\mathbf{v}$. Furthermore, it is known that $\mathbf{v}$ rests at a local minima, which implies that $\alpha > \beta$. Iteratively employing the update rule

$$z_i = \begin{cases} +1, & \text{if } h_i > 0 \\ -1, & \text{if } h_i < 0 \end{cases}$$

to the network lowers the energy of $\mathbf{z}$, and brings $\mathbf{z}$ closer to $\mathbf{v}$. This process continues until $\alpha = \beta$, at which point any change in $\mathbf{z}$ will lead to an increase in energy, therefore $\mathbf{z}$ is said to be in a stable configuration. This would also mean that in the process as $\alpha \to \beta$ so did $\mathbf{z} \to \mathbf{v}$. Since $\mathbf{v}$ just represents the state of the network with energy β, and $\mathbf{z}$ has energy β, that would imply that $\mathbf{z} = \mathbf{v}$. Thus the input is reconstructed.

Therefore, given any input that has not been learned by the network, the network will change that input, neuron-state by neuron-state, until the energy of that input reaches a local minima, corresponding to some learned input. At the local minima the network can no longer make a change to any neuron without incurring an increase in energy, thus the process comes to a halt, and the random input has been changed to a learned input.

Part 2: Restricted Boltzmann Machine

The following sections will first introduce what the Boltzmann Equation is and show how, utilising the this equation, Stochasticity is introduced to the Hopfield Network. Then it will be investigated how the Boltzmann Machine learns and how this differs from the Hopfield Networks learning. Finally the general architecture and restrictions underlying the Restricted Boltzmann Machine (RBM) will be explored and explained.

2.1 Boltzmann's Equations

We will now explore how one can leverage statistical mechanics in order to introduce stochasticity into a Hopfield Network, which will ultimately transform our model from a machine that could only generate data based entirely on previously memorised datasets, to a model that can actually understand the underlying distribution, and generate data according to this learned distribution.

Ludwig Boltzmann sought to understand the distribution of energy among a system of particles. Boltzmann discovered that the probability of finding a system in a specific state S_E with energy E is proportional to an exponential function of it's energy. More formally:

$$P(S_E) = \frac{1}{Z} e^{-\beta E}$$

Where Z is a normalising constant which can be calculated as the sum over all possible states, and β is the inverse temperature $\beta = \frac{1}{k_B T}$

$$Z = \sum_i e^{-E_i \beta}$$

One can draw extremely interesting insights from Boltzmann's discoveries here, which is that a state with higher energy is exponentially less likely to occur than a state with a lower energy. This explains why systems tend to settle into their lowest energy configurations, as they are exponentially more probable. Additionally, we can observe that Temperature plays a crucial role, as if the temperature increases the probability of finding high energy states increases, and conversely if Temperature decreases the probability of finding these high energy states decreases.

Linking this to what we observed with the Ising model, at low temperatures below the critical temperature T_c, it was observed that the system would most likely be found in a lower energy state where the spins are mostly aligned with their neighbours. Conversely, for temperatures above the T_c, it was observed that the system was more likely to be found in a higher energy state, where spins are misaligned with their neighbours.

2.2 Introducing Stochasticity to the Hopfield Network

In section 1.2, it was show that the neuron x_i was updated according to the weighted sum of all the neurons h_i connected to the neuron x_i, and the value for x_i was chosen in such a way as to decrease the total energy of the system. This update rule is deterministic in nature, as the value of h_i unambiguously determines the value for x_i. This is the first area where the Boltzmann Machine distinguishes itself from the Hopfield Network, as for the Boltzmann Machine the update rule makes probabilistic decisions based on the Boltzmann distribution:

$$x_i = \begin{cases} +1, & \text{if } P_{+1}(h_i) \\ -1, & \text{if } P_{-1}(h_i) \end{cases}$$

Using the Boltzmann equation stated prior, we well now derive the equation for the probability a neuron is in the +1 state (in this context we refer to this as switching the neuron on). We start by isolating that neuron and only considering the weighted connections to that neuron we will calculate the energy if it were to be switched on, and utilise that value to calculate the probability of switching the neuron on. Formalising this:

$$E_{\text{on}} = -\sum_{i \neq j} w_{ij} x_j + E_{rest}$$

$$E_{\text{off}} = \sum_{i \neq j} w_{ij} x_j + E_{rest}$$

Where E_{rest} represents the energy of the rest of the system excluding this neuron. It follows that the probability of this neuron being switched on $P_{\text{on}} = P(x_i = +1)$ is:

$$P_{\text{on}} = \frac{1}{Z} e^{-\beta E_{\text{on}}} = \frac{e^{-E_{\text{on}}\beta - E_{\text{rest}}}}{e^{-E_{\text{on}}\beta - E_{\text{rest}}} + e^{-E_{\text{off}}\beta - E_{\text{rest}}}}$$

$$P_{\text{on}} = \frac{e^{-E_{\text{on}}}\beta}{e^{-E_{\text{on}}/k_B T} + e^{-E_{\text{off}}\beta}} = \frac{1}{1 + \beta(e^{E_{\text{on}} - E_{\text{off}}})} = \frac{1}{1 + e^{-2\beta \sum_{i \neq j} w_{ij} xj}}$$

$$\therefore P_{\text{on}} = \frac{1}{1 + e^{-2\beta \sum_{i \neq j} w_{ij} xj}} = \sigma\left(\beta \sum_{i \neq j} w_{ij} xj\right)$$

Where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function. This clearly illustrates that the probability of the neuron x_i being switched on is the sigmoid function of the weighted sum of the neighbours. The sigmoid function converts a linear summation of terms into a function restricted to a range between 0 and 1 that

asymptotically approaches these boundaries and is symmetric about $y = 0.5$, where y represents the output of the sigmoid function. This clearly has a very elegant interpretation, as we can interpret the output of this function as a probability.

We use this sigmoid function when updating the state of each neuron, which means the system does not always make a decision that is guaranteed to decrease the total energy, as the neurons can sometimes increase the energy of the system with a probability that depends on the energy difference and the Temperature. The added stochasticity at the update step is crucial in differentiating the Hopfield Network and the Boltzmann Machine, as before the step we took in the energy landscape was taken deterministically as it was the step that minimises the energy, however, the Boltzmann Machine has a probability of escaping local minima and explore states that may not necessarily minimise the energy. The degree of stochasticity in the steps is controlled by the hyper-parameter Temperature, as one can observe it in the final equation for P_{on}.

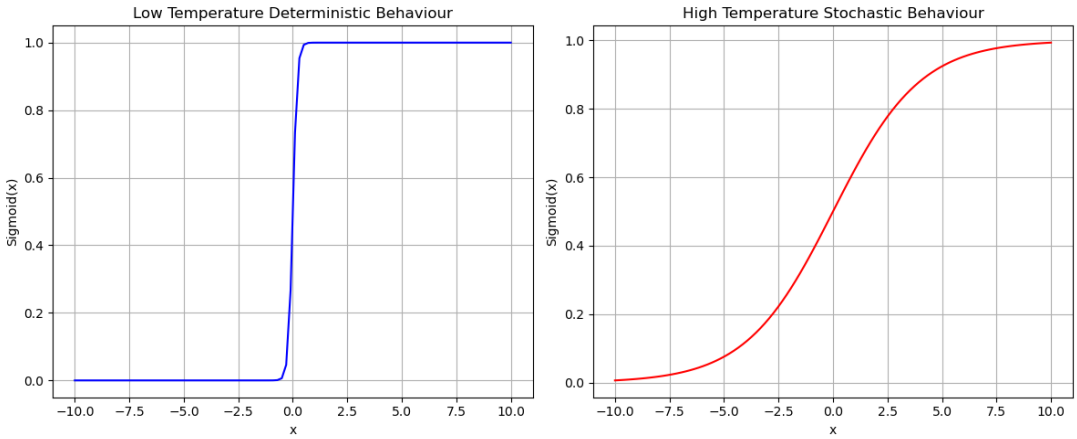

Figure 3: Sigmoid function output for Low vs High Temperatures

One can see in Figure 3 that the probability output of the sigmoid function for low temperatures is far more deterministic as the output is either 0 or 1 exactly, and is converging to a step function. However, for the high temperature scenario, the sigmoid is flattening out, indicating the output will be anywhere in the range between 0 and 1, and appears to be converging to a uniform distribution where the decisions made are completely random.

Connecting this now to the Ising model, we know that for temperatures that exceed the critical temperature T_c the model exhibits disordered spin behaviour. Our Boltzmann Machine accounts for this, as by tuning the temperature hyper-parameter the model will take more random steps in the energy space, and is far more likely to converge to a disordered state than an ordered state.

2.3 Training the Boltzmann Machine

We observe a training data set: $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N\}$ which we denote by Data.

The goal of our unsupervised learning task is to maximise the joint probability of the training data given the model's learned distribution, which is otherwise known as the likelihood of the data. We denote this as $P(\text{Data}|\Theta)$ where Θ represents the parameters of the model. Expressing this optimisation problem formally:

$$\Theta^* = \arg\max_{\Theta} P(\text{Data} \mid \Theta)$$

In order to solve this, we must first express $P(\text{Data}|\Theta)$ in a differentiable form:

$$P(\text{Data}|\Theta) = \prod_{n=1}^{N} P(\mathbf{x}_n)$$

We then take the logarithm of the multiplication, as this will convert the multiplication to a sum, allowing for easier future optimisation, as we will eventually be taking the derivative of the likelihood with respect to the parameters, in order to find the maximum value of the likelihood.

$$\begin{aligned}
\log P(\text{Data}|\Theta) &= \sum_{n=1}^{N} \log P(\mathbf{x}_n) && (1)\\
&= \sum_{n=1}^{N} \log \frac{1}{Z} e^{-\beta E(\mathbf{x}_n)} && (2)\\
&= \sum_{n=1}^{N} (-\log Z - \beta E(\mathbf{x}_n)) && (3)\\
&= -\beta \sum_{n=1}^{N} E(\mathbf{x}_n) - N \log Z && (4)
\end{aligned}$$

By investigating the simplified expression for the log-likelihood, we should update our parameters such that we minimise the total energy of our training data, and additionally minimise the partition function. Thinking about this intuitively, minimising the Energy of our training data has the effect of assigning our training patterns into deep energy wells in the energy landscape. By minimising the partition function we are assigning high energies to states that do not appear in our training set because the partition function is defined by summing over all available states.

We now derive the rule for updating the parameters, by differentiating the log-likelihood and setting it equal to zero (as this will find the maximum)

$$\frac{\partial}{\partial w_{ij}} \log P(\text{Data}|\Theta) = \frac{\partial}{\partial w_{ij}} (-\beta \sum_{n=1}^{N} E(\mathbf{x}_n) - N \log Z) \tag{5}$$

Splitting this problem into two parts, we solve the derivative of each sum term separately:

i)

$$\begin{aligned}
\frac{\partial}{\partial w_{ij}} (-\beta \sum_{n=1}^{N} E(\mathbf{x}_n)) &= -\beta \frac{\partial}{\partial w_{ij}} \sum_{n=1}^{N} \sum_{i \neq j} w_{ij} x_i x_j && (6)\\
&= \beta \sum_{n=1}^{N} \sum_{i \neq j} \frac{\partial}{\partial w_{ij}} w_{ij} x_i x_j && (7)\\
&= \beta \sum_{n=1}^{N} x_i x_j && (8)\\
&= \beta \, N \langle x_i x_j \rangle_{data} && (9)
\end{aligned}$$

ii)

$$\frac{\partial}{\partial w_{ij}}(-N \log Z) = -N \frac{1}{Z} \frac{\partial}{\partial w_{ij}} \sum_s e^{-\beta E_s} \tag{10}$$

$$= -\frac{N}{Z} \sum_s \frac{\partial}{\partial w_{ij}} e^{\beta \sum_{i \neq j} w_{ij} x_i \, x_j} \tag{11}$$

$$= -N\beta \sum_s x_i \, x_j \, \frac{1}{Z} \, e^{-\beta E_s} \tag{12}$$

$$= -\beta \, N \, \langle x_i x_j \rangle_{\text{model}} \tag{13}$$

Putting this all together we obtain the following gradient of the log-likelihood with respect to the weights:

$$\frac{\partial}{\partial w_{ij}} \log P(\text{Data}|\Theta) \propto \langle x_i x_j \rangle_{data} - \langle x_i x_j \rangle_{model}$$

$$\therefore \Delta w_{ij} \propto \langle x_i x_j \rangle_{data} - \langle x_i x_j \rangle_{model}$$

This equation is known as the contrastive Hebbian rule, and dictates how we train the Boltzmann Machine based on our training data. $\langle x_i x_j \rangle_{data}$ is known as the Hebbian term, and represents the expected correlation between neurons in the training data, and is directly analogous to the update rule for the Hopfield Network. Conversely, the term $\langle x_i x_j \rangle_{model}$ is known as the anti-Hebbian term, and captures the expected correlations between neurons by letting the model with it's current set of parameters run freely. By taking the difference between these two terms, we are subtracting the correlations generated by the model that do not match the training data, such that the model does not reinforce spurious correlations.

2.3 Restricting the Boltzmann Machine

The model explored in the prior sections is the generalised Boltzmann Machine with no hidden layers, where all the neurons are visible and form a fully connected graph. A hidden layer is a set of latent neurons that don't correspond to any physical element of our system, but have weighted connections to the visible units of our system. The hidden neurons capture greater complexity and patterns within the data, and introduce non-linearity to our model by their activation functions which will be elaborated on later.

We denote our visible neurons by $\mathbf{v} \in \mathbb{R}^p$ and the hidden neurons by $\mathbf{h} \in \mathbb{R}^q$. Additionally, the model includes biases vectors for each layer which denote as $\mathbf{a} \in \mathbb{R}^p$ and $\mathbf{b} \in \mathbb{R}^q$ where a_i and b_i represent the bias of the i'th unit of the visible and hidden layer respectfully. Lastly, we impose the restriction that units within the same layer may not be connected to one another.

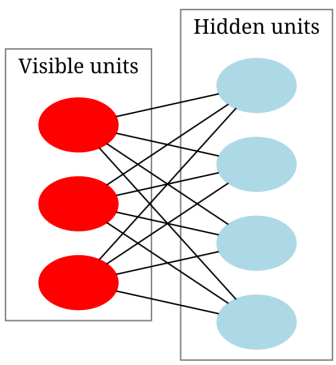

Figure 4: Simple RBM structure with one visible and one hidden layer

As a result, the energy of the RBM becomes:

$$E(\mathbf{v}, \mathbf{h}) = -\sum_i a_i v_i - \sum_j b_j h_j - \sum_{i,j} v_i W_{ij} h_j$$

And the contrastive Hebbian update rule becomes:

$$\Delta w_{ij} \propto \langle v_i h_j \rangle_{data} - \langle v_i h_j \rangle_{model}$$

Using the same calculus applied in section 2.3 when deriving the contrastive divergence rule, the dervitive of the log-likelihood of the data with respect to the bias terms can be derived as being:

$$\Delta a_i \propto \frac{\partial \log P(Data|\Theta)}{\partial a_i} = \langle v_i \rangle_{\text{data}} - \langle v_i \rangle_{\text{model}}$$

$$\Delta b_j \propto \frac{\partial \log P(Data|\Theta)}{\partial b_j} = \langle h_j \rangle_{\text{data}} - \langle h_j \rangle_{\text{model}}$$

Part 3: Training the RBM on Ising model configurations

In the following section the data generation and RBM training process will be discussed.

One of the most important things to consider when gathering samples is whether or not they are identically and independently distributed (iid). Therefore, the first 2 proceeding sections will deal with considerations pertaining to the independence of samples. The focus will be on how different boundary conditions and sampling methods effect the autocorrelation time of the system.

3.1 Investigating different Boundary Conditions

Section 3.1 introduces the concept of boundary conditions, what they are and how they effect the samples drawn from the Ising model. Firstly an explanation of what boundary conditions are and what they are used for will be provided. Thereafter, the effect different boundary conditions have on the Metropolis algorithm will be explored. Specifically how the boundary conditions affect the autocorrelation time of the algorithms and the finite size effects that they introduce into the simulation.

3.1.1 Boundary Conditions Explained

At the edge of the lattice, spin states do not possess all 4 neighbours, and therefore the general update rule for the Ising Model cannot be trivially applied to them. This is where boundary conditions are introduced as a set of rules on how to update the spin state of an Ising model that is located at the edge of the lattice. Furthermore, due to the discrepancy in the size of simulated lattices and real word lattices, finite size effects occur in the Ising simulations. These finite size effects can lead to inconsistent observations between the simulated system and real word experiments. Therefore, boundary conditions can also be imposed in order to mitigate finite size effects.

At T_c the autocorrelation time of the system increases exponentially due to critical slowing down. This means that sequential lattice configurations are become more correlated. This phenomena must be considered when samples are drawn from the lattices to make sure that the samples are iid.

3.1.2 Effects of Different Boundary Conditions

The main focus of the following section, mainly based on [3], will be to investigate how different boundary conditions affect the autocorrelation time of the Ising system close to the critical temperature ($\beta J = 0.44 \approx T_c$).

To measure the autocorrelation time the autocorrelation function is used. The autocorrelation function $\rho(k)$ is defined as the degree to which an observable $f : S \to \mathbb{R}$ (such as magnetisation M) at a given sweep n is correlated with it's value k sweeps away. This can be expressed as

$$\rho(k) = \rho(f(X_n), f(X_{n+k}))$$

.

It is expected that $\rho(k)$ will decay exponentially as $\rho(k) \approx e^{-k/\tau}$ where τ is the autocorrelation time. τ characterises roughly how many sweeps it takes for the samples to decorrelate.

However, autocorrelation time should not be the only consideration, since it is possible for a boundary condition to drastically reduce the autocorrelation time, but in doing so introduces finite size effects. Therefore, the binder cumulant values will be calculated for different boundary conditions at different lattice sizes.

The Binder Cumulant Value can be calculated as

$$U_L = 1 - \frac{\langle M^4 \rangle}{3\langle M^2 \rangle^2}$$

And represents a dimensionless value such that when: $L \to \infty$, then $U_L \to 2/3$ for $T < T_c$ and $U_L \to 0$ for $T > T_c$.

Given boundary conditions that make the system appear infinite, plots of U_L generated for different L-values should intersect close to T_c. The stability of the different graphs around T_c is a good indicator of finite size effects. That is, the closer the U_L values are to each other at T_c, for different lattice sizes indicates that the finite size effects are not as severe. The reason why this is the case is because if the boundary conditions do make the lattice behave more like an infinite than finite lattice, the more similar lattices of different sizes will appear. This similarity is captured in the insertion at T_c of plots for different binder cumulant values calculated at different lattice sizes.

Therefore, taking both the autocorrelation time and binder cumulant values into consideration the following boundary conditions are chosen to explore the different effects that they have on the Ising model:

1. **Periodic** - The lattice is wrapped around such that, all spins on the left edge observe their neighbouring spins to their left, the spins on the right edge. The spins at the top see their neighbouring spins above them, as the spins at the bottom.
2. **Antiperiodic** - has the same wrapping as periodic boundary conditions, but the spins are inverted. If a spin at an edge warps around it observes the opposite spin, from the spin it would have observed given periodic boundary conditions.
3. **Mirror** - A spin at an edge observes the previous spin in the opposite direction. Say a spin is at the left edge, then it will observe the spin to its right as the spin to its left.
4. **Fixed** - All the spins at edges see up spins.
5. **Free** - All edge spins do not see any spins apart form those that are in the lattice. There are effectively no boundary conditions.

In Figure 5 the binder cumulant values were calculated to investigate if different boundary conditions introduce finite size effects into the simulated system.

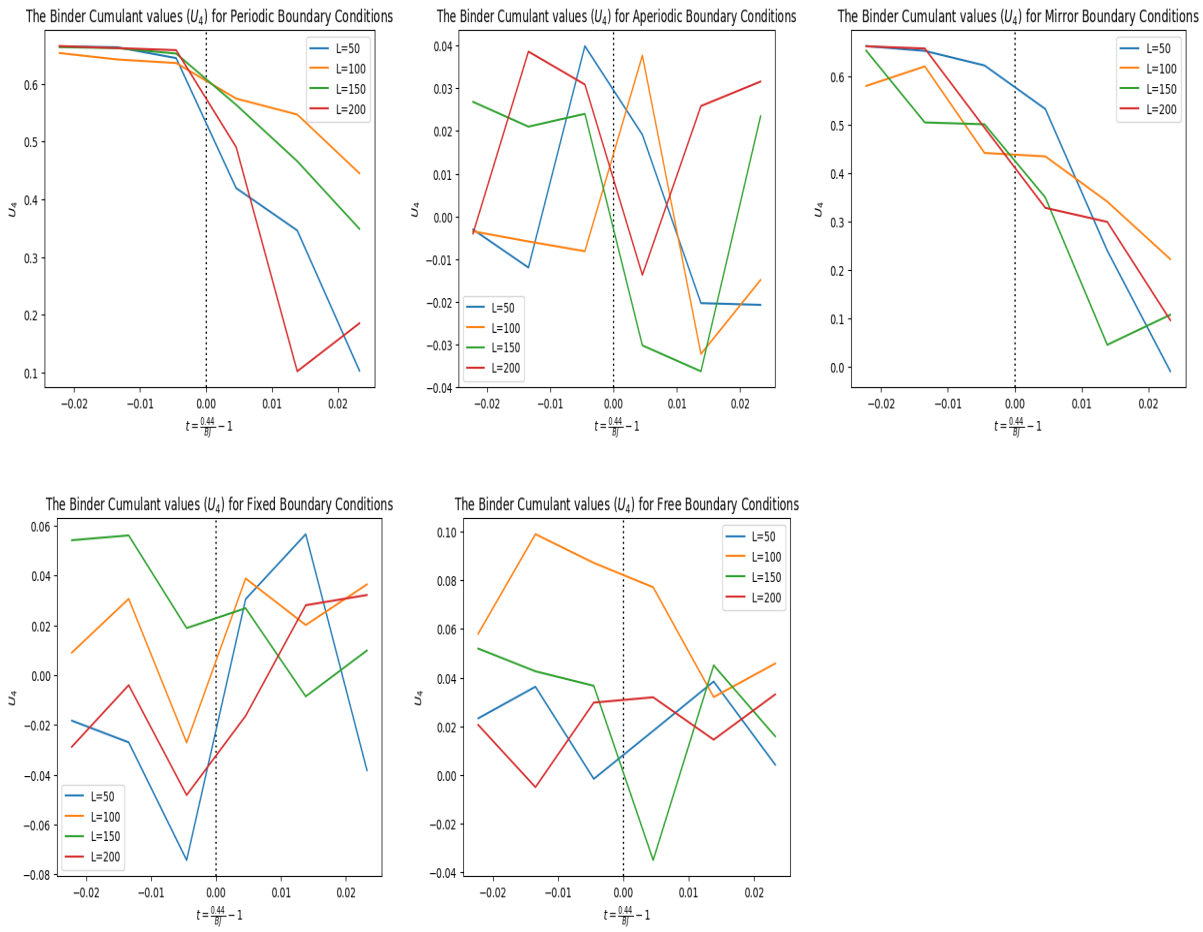

Figure 5: Graphs showing the binder cumulant values (U_L) at different lattice sizes for the following boundary conditions: (a) Periodic Boundary Conditions, (b) Antiperiodic Boundary Conditions, (c) Mirror Boundary Conditions, (d) Fixed Boundary Conditions, and (e) Free Boundary Conditions

From the various plots, in Figure 5, it can be observed that the only two boundary conditions that do not introduce to much finite size effects, are the Periodic and Mirror Boundary Conditions. In all other cases the plots for different lattice sizes do not appear to be nearly as stable for the 2 before mentioned boundary conditions. Additionally, only plot (a) and (c) depict the expected behaviour of U_L for large and small T's, that is $U_L \to 2/3$ for $T < T_c$ and $U_L \to 0$ for $T > T_c$.

Figure 6 shows the results yielded from investigating the effect that different boundary conditions have on the autocorrelation time of the system at $\beta J = 0.44$.

From Figure 6 it can be seen that again Periodic and Mirror Boundary Conditions stand out, as the only 2 boundary conditions that have $\tau > 10$.

At this point the question, of what boundary condition to use during sampling, can be given a somewhat reasonable answer. Firstly note that even though a $\tau = 7$, such as the Antiperiodic, Fixed and Free Boundary Conditions possess, these boundary conditions would not produce good samples for the Ising Model. The reason being that according to the their respective binder cumulant value plots, they introduce severe finite size effects into the simulated system. Therefore, they cannot be considered as possible boundary conditions. When considering between the mirror and periodic boundary conditions, the periodic boundary condition comes on top due to its binder cumulant values being more stable close to T_c as well as its autocorrelation time being shorter than the that of the mirror boundary condition.

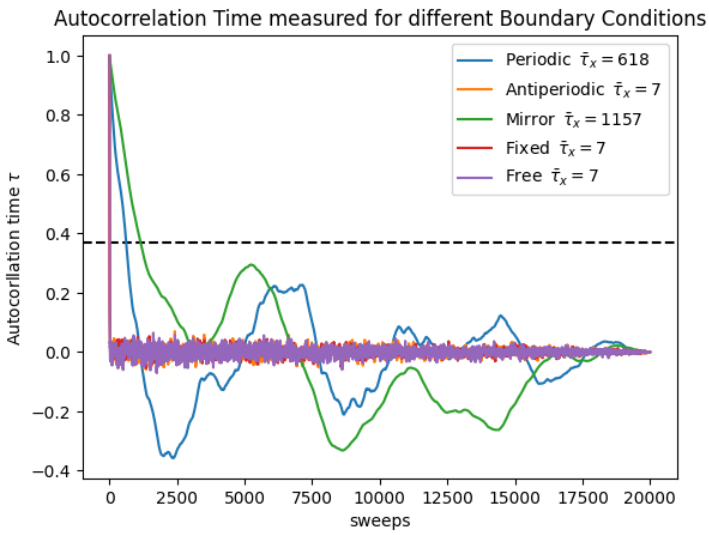

Figure 6: The autocorrelation time for different boundary conditions

3.2 Investigating the efficiency of Cluster vs Single spin-flip algorithms

3.2.1 Motivating the use of Cluster Algorithms

In Onsager's paper titled "*Crystal Statistics A Two-Dimensional Model with an Order-Disorder Transition*" [4] it was shown that in the limit of large lattices $L \to \infty$, the average magnetisation per spin can be expressed exactly for $T < T_c$ as:

$$\lim_{L\to\infty} E[|m|] = (1 - (\sinh 2\beta J)^{-4})^{\frac{1}{8}}$$

Additionally, for $T > T_c$ it was found that the system had a net magnetisation per site of zero, and hence the $E[|m|]$ serves as an order parameter for the phase transition:

$$\lim_{L\to\infty} E[|m|] = 0$$

This results in a continuous phase transition at the critical temperature T_c, and as a result of this one can observe divergent behaviour in the correlation functions at the critical temperature. At T_c, the correlation length ξ which represents the characteristic length at which two spins are correlated across the lattice, diverges according to:

$$\xi \sim |T - T_c|^{-\upsilon}$$

Where υ is a critical exponent for the system. This divergence results in what is known as critical slowing down, and is because of the diverging correlation length a change in 1 spin after flipping it can have an effect on a very large amount of spins far away. From this it follows that the auto-correlation time τ is:

$$\tau \sim \xi^z = |T - T_c|^{z\,\xi}$$

Where z is a dynamic critical exponent, that characterises the rate of convergence of the auto-correlation time relative to the correlation length. This relationship displays how the auto-correlation time also diverges, meaning the time for our simulation to start generating independent samples starts becoming infeasible close to T_c. Understanding that z is not a universal exponent for the Ising model is crucial, as

if we use a more efficient algorithm such as a cluster flip algorithm, we can decrease the magnitude of z and therefore acquire more optimal auto-correlation times near the phase transition.

3.2.2 The Wolff Algorithm

The Wolff algorithm is a cluster flip algorithm designed to improve the efficiency of our simulation by flipping entire clusters of aligned spins in one step, instead of the individual spin flips as before. The algorithm works as follows:

1) Randomly select a spin $s_i = \sigma_0$ from the lattice

2) All the nearest neighbours aligned with the seed spin as follows: $s_j = \sigma_0$ are added to the cluster with probability:

$$p_{ij} = 1 - e^{-2\beta J}$$

3) The algorithm recursively explores all the eligible neighbouring spins of the newly added spin, and repeats steps (2) until all available spins have been visited.

5) All the spins in the cluster are then flipped.

6) Repeat steps 1 to 5 for an arbitrary number of steps.

3.2.3 Comparing the Wolff Algorithm with the Metropolis Algorithm

In order to decide which algorithm to utilise near criticality, an investigation was conducted into the efficiency of the cluster flip Wolff and single-spin flip Metropolis algorithm using the autocorrelation function $\rho(k)$ and the autocorrelation time τ as described in section 3.1.2.

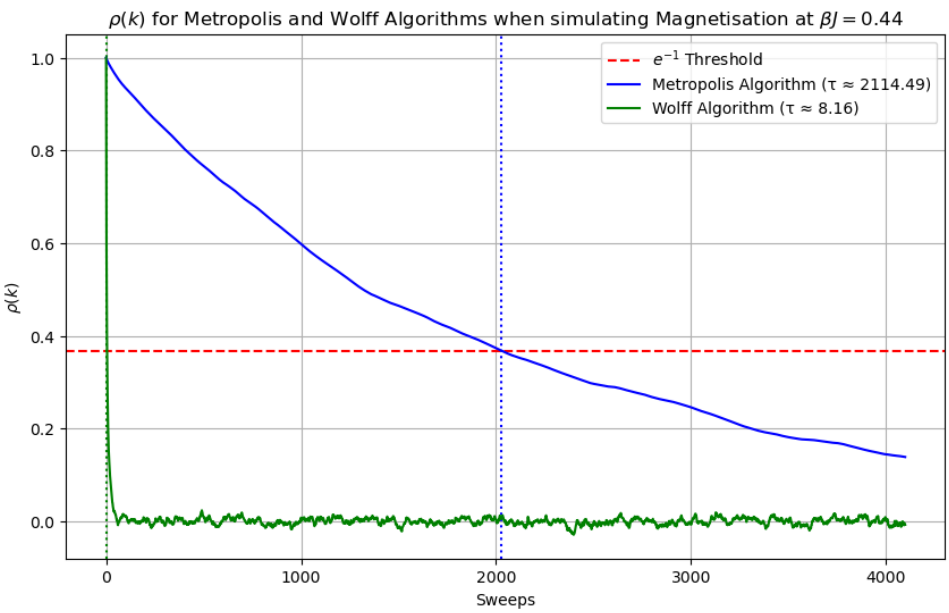

Figure 7: The autocorrelation function $\rho(k)$ of the Magnetisation vs sweeps at the critical temperature T_c

One can observe that at criticality the Wolff algorithm takes significantly fewer sweeps to decorrelate compared to the Metropolis algorithm, with respective autocorrelation times of $\tau_{\text{Wolff}} \approx 8.16$ and $\tau_{\text{Metropolis}} \approx 2114.49$. This motivates the use of the Wolff algorithm for simulating the dataset of spin configurations close to T_c, as it takes significantly less time in sweeps for the configurations to decorrelate.

3.3 Generating the Training Dataset

Generating a diverse and rich dataset is crucial for the performance of the RBM. A dataset D_T for temperature T consisting of $M = 20\,000$ observations was generated using the Wolff algorithm with periodic boundary conditions, due to respectively low autocorrelation times and minimised finite size effects.

Each element of the dataset is defined as:

$$d_{ij} = \{1, 0\}, \text{ for } i = 1, 2, ..., M \text{ and } j = 1, 2, ..., L^2$$

In which 1 represents a particle with spin-up, and 0 represents a particle with spin-down. The data is stored row-wise such that each row represents a flattened representation of a sampled lattice, with a length of L^2. The final shape of the dataset D_T is $M \times L^2$

A dataset D_T was generated for 3 Temperatures $\beta J = 0.36$, $\beta J = 0.44$, and $\beta J = 0.52$, for a lattice size of 64×64. In order to ensure that each consecutive sample drawn using MCMC methods is independent and uncorrelated, after the system had equilibrated each sample was separated by a pre-specified number of sweeps. This process is known as thinning, and the autocorrelation time τ is used to determine how many sweeps to run before sampling a new configuration.

Part 4: Training and Evaluating the RBM

4.1 Training the RBM

Hyper-Parameters

When training any machine learning model, there are always a abundance of different parameters that do not only describe the values learned by the model, but also describes more general structures in which the model was able to learn those parameter values. In the case of the Restricted Boltzmann Machine the parameters that the model can learn are $\mathbf{W}$, $\mathbf{a}$, and $\mathbf{b}$, but in addition there are also $\mathbf{h}$, γ the learning rate, and k the number of Gibbs sampling steps that can not be learned by the RBM but are user specified parameters, hyper-parameters.

The goal of the this section will be to determine what hyper-parameters effect the model in a positive way. The effects of $\mathbf{h}$, γ, and k on the model will be independently analysed. It should be noted that it is assumed that these parameters are independent from each other and that therefore, the model trained with the conjunction of best hyper-parameter values will perform the best.

Before proceeding it is import to define what is meant by "positive". In the machine learning, there are goals that models aim to achieve, by either minimizing or maximizing some value. Any change in the models hyper-parameter values that leads to a model being closer to its aim, is said to be a positive change. In the case of RBM the dual aim is to decrease the cross entropy of the model, which is equivalent to reducing the reconstruction error, and reducing the pseudo log-likelihood of the mode

Cross Entropy

In the context of the RBM the Cross Entropy (CE) is equal to the reconstruction error of the model. The general overview of the algorithm to calculate the reconstruction error goes as follows:

Keep all parameters constant($\mathbf{W}$, $\mathbf{a}$, $\mathbf{b}$).

1.) Get input $\mathbf{v}^i$.

2.) Find the hidden layer from the visible layer. That is, calculate $P(\mathbf{h}^i|\mathbf{v}^i)$.

3.) Reconstruct the visible layer $\mathbf{v}^r$, with the calculated $\mathbf{h}^i$. That is, calculate $P(\mathbf{v}^r|\mathbf{h}^i)$.

4.) Find the Cross Entropy between $\mathbf{v}^i$ and $\mathbf{v}^r$:

$$\text{cross entropy} = -\frac{1}{N}\sum_{i}^{N}\left(\mathbf{v}_i^i \cdot \ln(\mathbf{v}_i^r) + (1-\mathbf{v}_i^i)\cdot \ln(1-\mathbf{v}_i^r)\right)$$

Cross Entropy measures the distance between $\mathbf{v}^i$ and $\mathbf{v}^r$, the more similar they are the lower CE will be. Therefore, the more accurately the parameters allow the RBM to reconstruct the visible layer from the hidden layer, the better the RBM has learned the data and its distribution.[5]

Pseudo Log-Likelihood

Earlier in section 2.3 it was shown that the log-likelihood is equal to:

$$-\beta\sum_{n=1}^{N} E(\mathbf{x}_n) - N\log Z$$

However, problems arise when trying to calculate Z. For large systems, Z is intractable, and therefore the log-likelihood cannot be calculated. That is, because to calculate Z it is needed to calculate all possible visible configuration, which is extremely expensive for large lattice sizes.[5]

Thus a pseudo log-likelihood is used, which approximates the true log-likelihood, by calculating the log probability of observing a the state of a single spin given all the other spin states at a given instance. This can be formally expressed as:

$$\log P(v) \approx \sum_{i=1}^{n} \log P(v_i|v_{\setminus i})$$

Validation Set

Before continuing, a quick explanation of what a validation set is must be given. When training a model, the whole dataset is fed to the model multiple times. Each time the whole dataset has been fed to the model it is said that 1 epoch has passed. Due to the fact that the data is passed through the model multiple times, it is possible for the model to learn the data to well. Therefore, the model is tested and evaluated on a validation set. After each epoch, the model is then given unseen data, on which its weights are not updated, and it then calculates the CE and pseudo log-likelihood. This then represents an unbiased measure of how well the model has actually learned the data.

For training the RBM a training set of 20000 data points is used, and a validation set of 2000 data points. The data was divided into batches of size 128, and after each batch the parameters are updated with a learning rate of $\gamma = 0.0001$ (initially).

4.1.1 Model Architecture

The model is to be implemented in Python, using a combination of Numpy and Tensorflow. Numpy and Tensorflow offer high levels of optimisation in terms of their memory management and computation time, which is crucial for the task of building an Neural Network over many epochs.

The visible layer of the model is constructed from L^2 units where $L = 64$. This mimics the behaviour of the Ising model as each visible unit x_{ij} is defined as:

$$x_{ij} = \{0, 1\}, \text{ for } i = 1, 2, ..., L \text{ and } j = 1, 2, ..., L$$

Where 0 is representing spin-down, and 1 spin-up, and each visible unit effectively maps to a lattice sight.

The hidden layer of the model provides a latent representation of the data it was trained on, allowing the model to capture complex relationships and patterns within the data. The following points were important to consider when deciding the number of hidden units to utilise:

- **Model Complexity:** The more hidden units the model possesses, the more capable the model becomes at capturing complex relationships within the training data. This means the model will be more effective at capturing spin correlations and interactions.
- **Overfitting:** The concept of overfitting is crucial to building machine learning models which generalise effectively. The principle is that as we increase the complexity of the model (by say increasing the number of hidden units), the model becomes so flexible that it actually starts modelling the statistical noise within the dataset. As a result of this, one has to be cautious not to choose too many hidden units.

Therefore, an important consideration when training a good RBM is the number of hidden units. Additionally, the number of hidden units is the greatest contributor to computational cost and will therefore be tested first. For if a smaller number of hidden units provided better results, then that would reduce the training time for all other subsequent models.

To investigate the effect of the size that the number of hidden units have on CE and pseudo log-likelihood 3 different models are trained with $\mathbf{h} = \{\mathbf{v}/2, \mathbf{v}, 2\mathbf{v}\}$, at $T = \{1.9200, 2.2690, 2.7732\}$ respectively, where $\mathbf{v}$ is the number of visible units (4096). The following graphs are obtained:

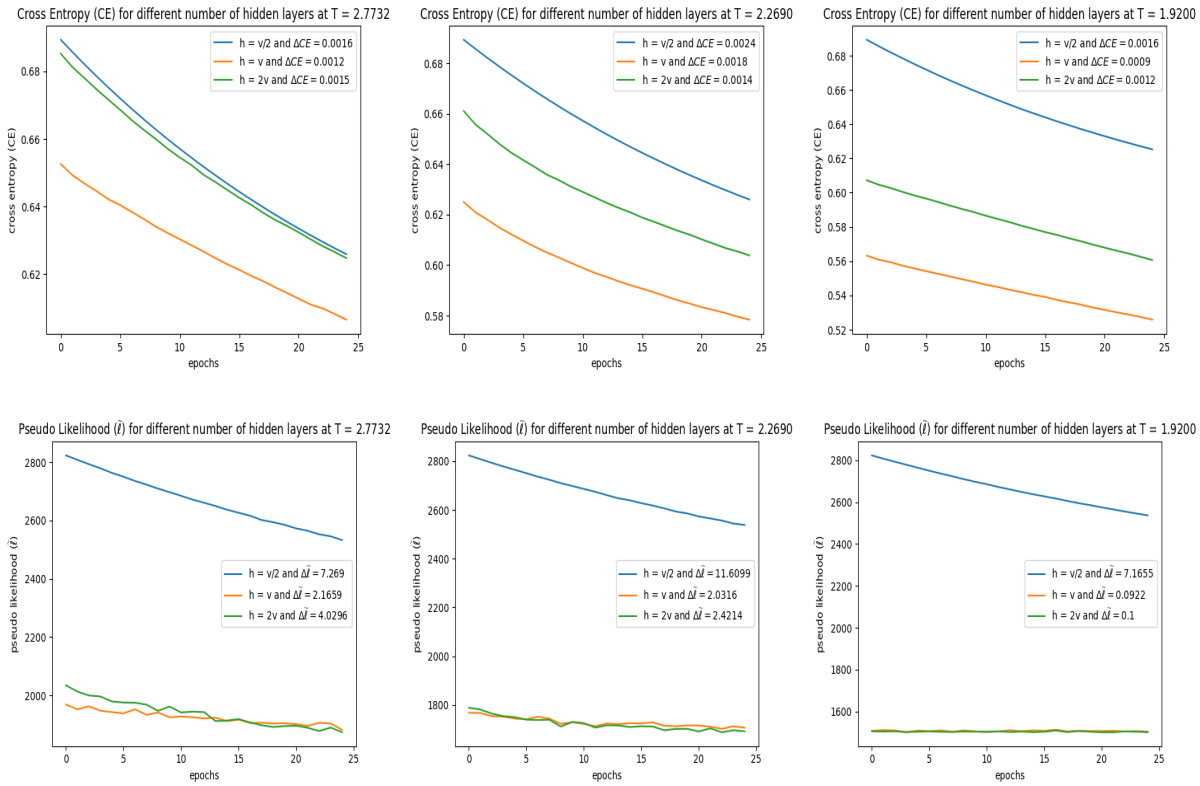

Figure 8: Graphs showing effect that different number of hidden units ($\{\mathbf{v}/2, \mathbf{v}, 2\mathbf{v}\}$) have on the training of the RBM over 25 epochs. (Top) Cross Entropy over the epochs, (Buttom) Pseudo Log-Likelihood over epochs.

Considering the results of the CE plots, the most significant result is that $\mathbf{h} = \mathbf{v}$ produces the lowest CE values, which means that when the number of hidden units is equal to the number of visible units the RBM has the lowest reconstruction error. The gradient of CE for the different values of $\mathbf{h}$ across the

different values of T, also seems to stay relatively consistent. With the gradient between the plots at the same temperature for different $\mathbf{h}$ values, appear to approximately decrease with the same rate.

The reason to why $\mathbf{h} = \mathbf{v}$ yields the lowest reconstruction error, is due to the fact that $\mathbf{h} = 2\mathbf{v}$ model is most likely too complex and therefore struggles to reproduce the input image on the first. The opposite is true of the $\mathbf{h} = \mathbf{v}/2$ model, which it would seem is not complex enough and therefore struggles to reproduce the input image from the start, leading to a greater reconstruction error.

Furthermore, there seams to be an increase in CE as temperature decreases for the $\mathbf{h} = \mathbf{v}/2$ model. This is most likely the case because at high temperatures, spins are randomly orientated, and that the expected value for each spin state is closer to 0. At T_c, spins are no longer randomly orientated, but the expected value of each spin state is still not too large. Only at small temperatures, when almost all the spins are up or down, does the expected value of each spin state diverge away from 0 to either 1 or -1. Therefore, when a model randomly reconstructs the input data, the closer the expected value of each spin state is to 0, the more spins it will coincidently assign the correct spin configuration.

Moving to the the pseudo log-likelihood($\widetilde{\ell}$) plots. The only things to note is that (1) the $\mathbf{h} = \mathbf{v}/2$ model has the highest $\widetilde{\ell}$ over all temperatures, (2) the change in $\widetilde{\ell}$ over epochs for the $\mathbf{h} = \mathbf{v}$ and $\mathbf{h} = 2\mathbf{v}$ models decreases as temperature decrease, and that at low temperatures $\widetilde{\ell}$ appears constant.

The large value of $\widetilde{\ell}$ for the $\mathbf{h} = \mathbf{v}/2$ model indicates that the RBM is struggling to learn the patterns present in the data. This is again mostly due to the fact that the RBM only has 2048 hidden units, and therefore struggles to capture the more complex patterns in the data.

Since $\widetilde{\ell}$ is an indication of how well the model is able to learn the data, thus from Figure 8 it is observed that the model does learn the patterns at a higher temperature, but at a low temperature $\widetilde{\ell}$ plateaus and the larger models struggle to learn anything. This is an indication of overfitting. The two larger models have to many parameters and are thus overfitting on the training data, while the smaller model is not overfitting, since it continues to decrease. The reason why the larger models overfit less at higher temperatures might be due to the complexity of the spin configurations, since at $T = 2.7732$ and $T = 2.2690$ there are a lot more variation in spin configuration, that is a lot of spins are spin up and spin down in a single lattice. While at low temperatures the lattice starts spins start to align, leading to most less variation between spins and this is a much easier pattern for the RBM to learn, thus it overfits more easily compare to the higher temperatures.

Even though the larger models had smaller CE they both seemed to severely overfit the data, thus smaller architectures will be investigated. The only architecture that didn've overfit was the smallest one, therfore the RBM was again trained with smaller numbers of hidden units. The new architectures with $\mathbf{h} = \{2048, 2400, 2800\}$ is trained for 20 epochs at the 3 different temperatures producing the following plots in Figure 9.

From Figure 9 the same results are observed as before, the more complex models tend to have a lower CE, but overfit with the $\widetilde{\ell}$. The only architecture that actually showed improvement in $\widetilde{\ell}$, over the epochs, was the smallest architecture. Once again even smaller model architectures will be investigated to see if models smaller than $\mathbf{h} = 2048$ can produce better results both in CE and $\widetilde{\ell}$. Therefore, the new architectures with $\mathbf{h} = \{400, 900, 1100\}$ are trained for 20 epochs at the 3 different temperatures producing the following plots in Figure 10.

From Figure 10 it is observed that the model with $\mathbf{h} = 900$ appeared to perform the best overall, since it starts with a generally low and decreases at a good rate relative to the other architectures. Furthermore, when comparing $\mathbf{h} = 900$ to the larger models with $\mathbf{h} = 2048$ and up, the smaller model seems to reduce the CE faster than the larger models. This might be due to the fact that the smaller model has fewer parameters to train and can therefore focus more on only improve a few parameters. It seems that $\mathbf{h} = 900$ is the sweet spot between a models ability to accurately capture and reconstruct the input data by not having to few parameters leading to suboptimal reconstructions such as $\mathbf{h} = 400$, but not having to much which would introduce to much complexity, such as $\mathbf{h} > 900$. Additionally its $\widetilde{\ell}$ is the only $\widetilde{\ell}$ that improves over all 3 temperatures, even though it is only slight improvements.

Due to time constraints and the time it takes to train these various models (approximately 12 hours), it

was decided that for subsequent models will be trained with $\mathbf{h} = 900$.

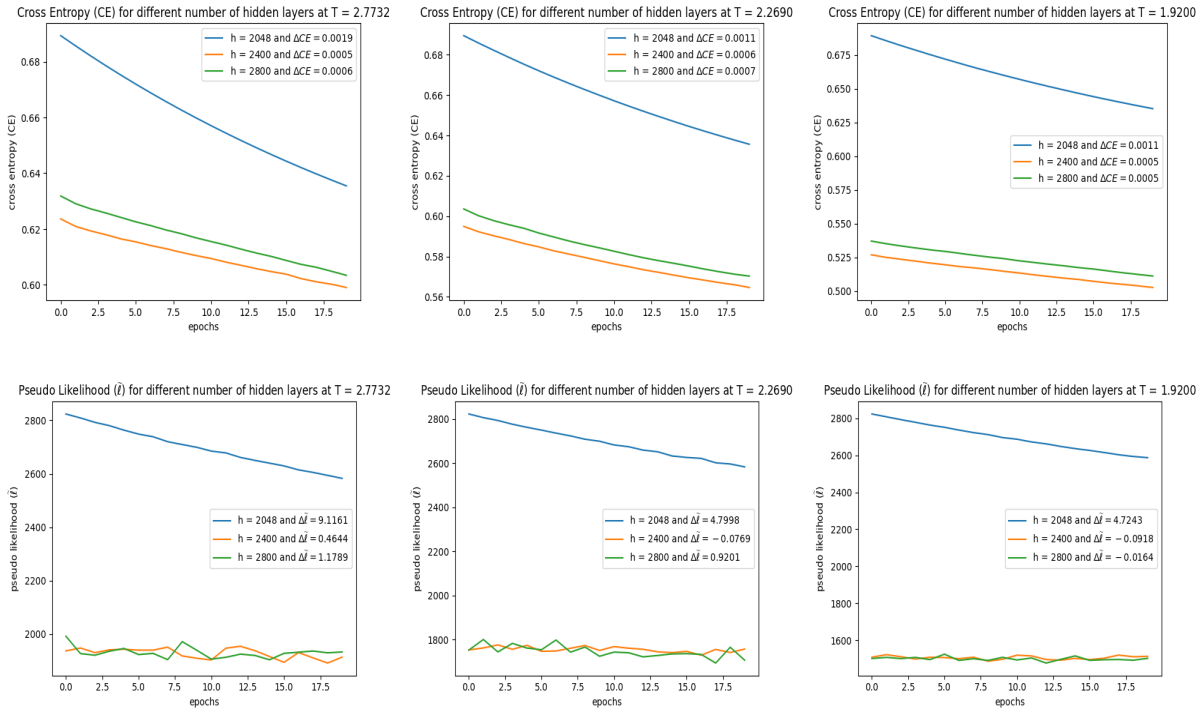

Figure 9: Graphs showing effect that different number of hidden units ($\{2048, 2400, 2800\}$) have on the training of the RBM over 20 epochs. (Top) Cross Entropy over the epochs, (Buttom) Pseudo Log-Likelihood over epochs.

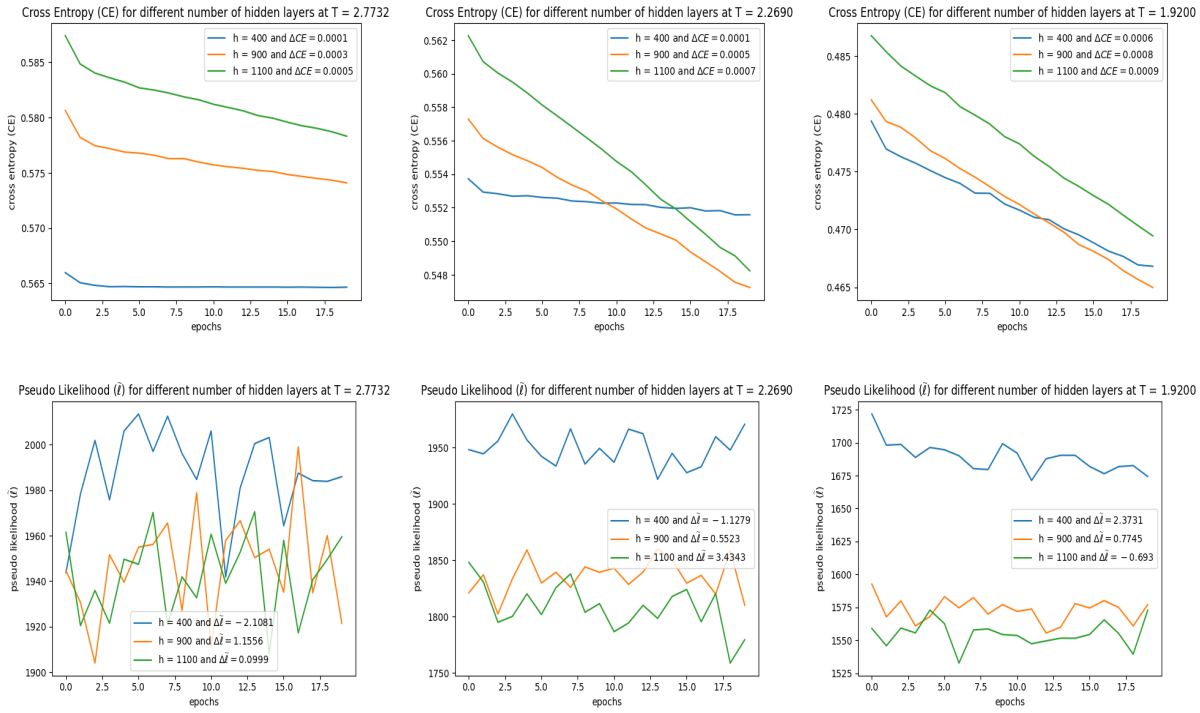

Figure 10: Graphs showing effect that different number of hidden units ($\{400, 900, 1100\}$) have on the training of the RBM over 20 epochs. (Top) Cross Entropy over the epochs, (Buttom) Pseudo Log-Likelihood over epochs.

4.1.2 Sampling new Spin Configurations

During and after the RBM's training, the parameter values of the weights and biases are calculated, and therefore new states can be generated to evaluate how well the model performs at generating new observations of the Ising model. This is utilized to calculate the Cross Entropy and reconstruction error. In this investigation, Gibbs sampling was utilised to sample from the models learned distribution.

Gibbs Sampling

The restrictions applied to the original Boltzmann Machine have an extremely powerful consequence, which allows for Gibbs Sampling to be used after the model has trained it's weights and biases. Because the model is restricted in such a way that there are no inter-connections between any units of the same layer for both the visible and hidden layer, it follows that the visible units are all conditionally independent of one another given the hidden layer is known. Conversely, it is true that the elements of the hidden layer are conditionally independent of one another given the visible layer is known. To express this mathematically for the probability of sampling a spin up state from the visible layer:

$$P(v_i = +1 \mid \mathbf{h}, v_j) = P(v_i = +1 \mid \mathbf{h}) \text{ for all } i \neq j \tag{14}$$

$$= \sigma\left(\beta\left(\sum_j h_j w_{ij} + a_i\right)\right) \tag{15}$$

Additionally, for the hidden layer it follows:

$$P(h_j = +1 \mid \mathbf{v}, h_i) = P(h_j = +1 \mid \mathbf{v}) \text{ for all } i \neq j \tag{16}$$

$$= \sigma\left(\beta\left(\sum_i v_i w_{ij} + b_j\right)\right) \tag{17}$$

Due to the conditional independence between layers, it follows that Gibbs sampling can be used to generate new lattice configurations. Gibbs sampling is a special case of the Metropolis Hastings algorithm, and is a MCMC algorithm used for sampling from multivariate probability distributions by splitting them up into separate conditional distributions which are simpler to sample from. In the case of the RBM sampling problem, we want to sample from $P(\mathbf{v}, \mathbf{h})$, however this distribution is not known, and instead we utilise the known conditional distributions formalised prior: $P(\mathbf{v} \mid \mathbf{h})$ and $P(\mathbf{h} \mid \mathbf{v})$.

Gibbs Sampling Algorithm

Given the p-dimensional initial visible vector $\mathbf{v}^{(0)} = (v_1^{(0)},\ v_2^{(0)},\ \ldots,\ v_p^{(0)})$

1. Update the hidden units by holding the visible units as fixed, and sampling the hidden unit h_j from the Bernoulli distribution:

   (a) Calculate the probability of h_j being +1, given the current state of the visible layer $\mathbf{v}^{(0)}$:

   $$P(h_j = 1 \mid \mathbf{v}^{(t)}) = \sigma\left(\beta\left(\sum_{i=1}^{n} v_i^{(t)} w_{ij} + c_j\right)\right)$$

   (b) Sample h_j from a Bernoulli distribution using this probability:

   $$h_j^{(0)} = \begin{cases} 1 & \text{with probability } P(h_j = 1 \mid \mathbf{v}^{(0)}) \\ 0 & \text{with probability } 1 - P(h_j = 1 \mid \mathbf{v}^{(0)}) \end{cases}$$

Where:

$$P(h_j = +1 \mid \mathbf{v}^{(0)})) = \sigma\left(\beta\left(\sum_i v_i^{(0)} w_{ij} + b_j\right)\right)$$

2. Once all the hidden units have been sampled forming $\mathbf{h}^0$, the hidden units are then held as constant and the visible units are updated. The visible unit v_i is updated in a similar way as the hidden units were updated:

   (a) Calculate the probability of v_i being active (1), given the current state of the hidden layer $\mathbf{h}^{(0)}$:

   $$P(v_i = 1 \mid \mathbf{h}^{(t)}) = \sigma\left(\beta\left(\sum_{j=1}^{m} h_j^{(t)} w_{ij} + b_i\right)\right)$$

   (b) Sample v_i from a Bernoulli distribution using the probability:

   $$v_i^{(1)} = \begin{cases} 1 & \text{with probability } P(v_i = 1 \mid \mathbf{h}^{(0)}) \\ 0 & \text{with probability } 1 - P(v_i = 1 \mid \mathbf{h}^{(0)}) \end{cases}$$

   Where:

   $$P(v_i = +1 \mid \mathbf{h}^{(\mathbf{0})}) = \sigma\left(\beta\left(\sum_i h_j^{(0)} w_{ij} + a_i\right)\right)$$

3. Repeat this process k times by circling back to step 1 by sampling new hidden units given the updated visible units.

Using the Gibbs sampling method, it becomes extremely efficient to sample new visible configurations of this Ising model, and after a sufficient number of steps k one is effectively sampling from the target distribution $P(\mathbf{v}, \mathbf{h})$

The question then becomes, what is the optimal number of Gibbs sample steps needed to reduce CE and improve the models performance? That is, how does k effect the models performance?

To test the effect that k has on the models performance, 3 different k values are selected ($k = \{1, 3, 5\}$) and the models are then ran for 20 epochs with these different k values at the same temperatures as before. This produces the following figure.

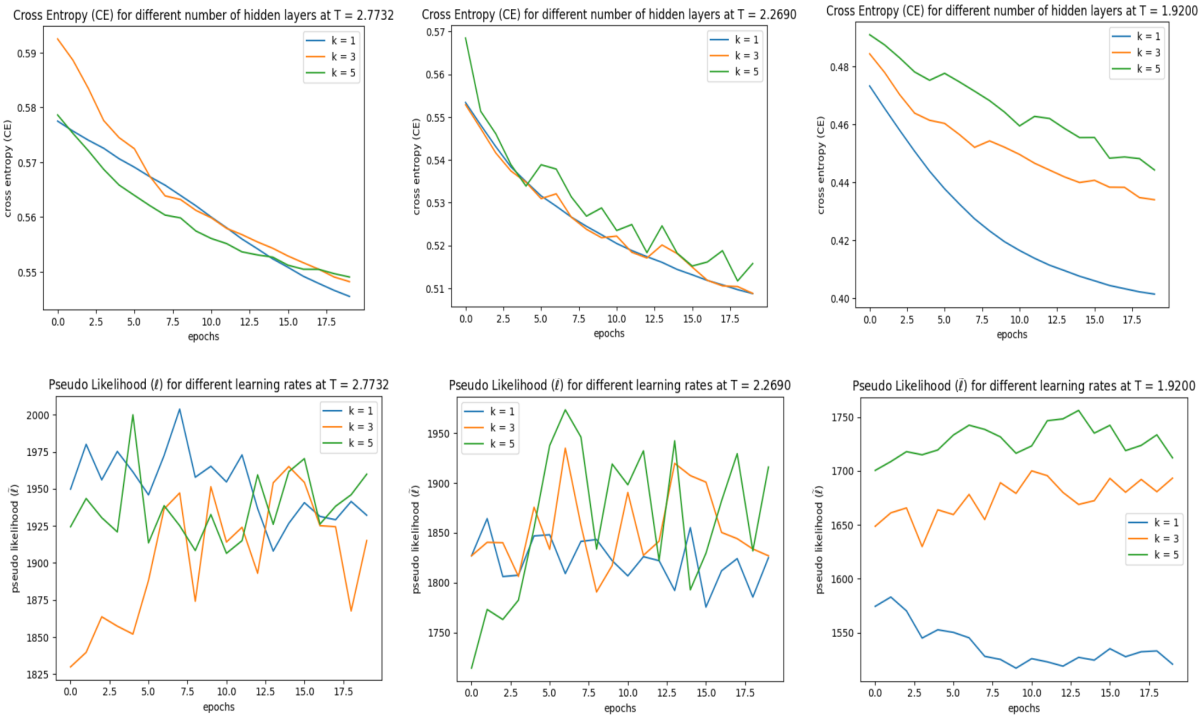

Figure 11: Graphs showing effect that different k values ($\{1, 3, 5\}$) have on the training of the RBM over 20 epochs. (Top) Cross Entropy over the epochs, (Buttom) Pseudo Log-Likelihood over epochs.

One of the first things to note is that $k = 1$ has the smoothest CE line, while both $k = 3$ and $k = 5$ have a lot more jagged lines. The jaggedness of these lines also seem to increase at $T \approx T_c$, this is due to the fact that the lattice configurations at that temperature are very complex in relation to lower and higher temperature lattice configurations, leading to more uncertainty and error in the reconstructed lattice configuration. An additional thing to note is that the jaggedness increase between $k = 3$ and $k = 5$ at T_c, which means that the large k is, the more variance there is in the reconstruction error. This might be due to the fact that CE is calculated with the conditional probabilities of $P(\mathbf{v} \mid \mathbf{h})$ and $P(\mathbf{h} \mid \mathbf{v})$ which, given that the model has not learned the true probability distribution, introduces errors by multiplying probabilities that are not precisely that of the true distribution, thereby introducing variance into the CE. Therefore $k = 1$ has the least variance in reconstruction error because it only does one Gibbs sampling step.

A further thing to note is that the curves for $k = 3$ and $k = 5$ are below $k = 1$ at high temperatures, and starts to move up as temperature increases. This again is due to the random orientation of spins at high temperatures. When multiple steps are computed the error introduce by the repeated sampling of the incorrect distribution, which might lead some spins to flip when they should not, does not introduce to much reconstruction error since the spins are randomly flipped to begin with. Another random flip or 2 won't do to much harm. Compared to $T \approx T_c$ and $T < T_c$ where spins are a lot more ordered, meaning that a random flip due to repeat Gibbs samples introduces much more reconstruction error into the system.

When looking at the $\widetilde{\ell}$, the only k that ends with a lower pseudo log-likelihood value then it started with is $k = 1$, that is most likely because the reconstructed input is the most similar to the actual input, since it undergoes the least amount of Gibbs steps. Therefore, any proceeding model will be trained with $k = 1$.

4.1.3 Applying Contrastive Divergence

Learning Rate

As discussed in section 2.3, the equation for contrastive divergence was formulated as:

$$\Delta w_{ij} \propto \langle v_i h_j \rangle_{data} - \langle v_i h_j \rangle_{model}$$

Which when calculated for all weights w_{ij} determines how the weight parameters should all be updated to optimise the model (by maximising the likelihood of the training data). In this investigation, the variant known as k-step contrastive divergence (CD-k) is used to train the RBM, which utilises k-steps of Gibbs sampling in order to approximate this gradient using the training dataset.

The updates for the parameters are as follows:

$$\Delta w_{ij} = \gamma \left(\langle v_i h_j \rangle_{\text{data}} - \langle v_i h_j \rangle_{\text{model}} \right)$$

$$\Delta a_i = \gamma \left(\langle v_i \rangle_{\text{data}} - \langle v_i \rangle_{\text{model}} \right)$$

$$\Delta b_j = \gamma \left(\langle h_j \rangle_{\text{data}} - \langle h_j \rangle_{\text{model}} \right)$$

Where γ is the learning rate hyper-parameter. The learning rate controls how large or small the magnitude of the parameter update should be, which controls how fast or slow the model learns.

Choosing the correct γ is of great importance, for if γ is to small, the model's weights might get trapped in a suboptimal local minima leading to the model converging to fast, where as if γ is to large the model might jump out of an more optimal local minima and behave more erratic and fail to converge. Therefore, before continuing it is important to investigate the effect of different learning rate on the CE and $\widetilde{\ell}$.

RBM's with $\mathbf{h} = 900$ and $k = 1$ are trained with different learning rates at the 3 different temperatures. The CE and $\widetilde{\ell}$ for the different models are measured accross 25 epochs, producing the plots in Figure 11.

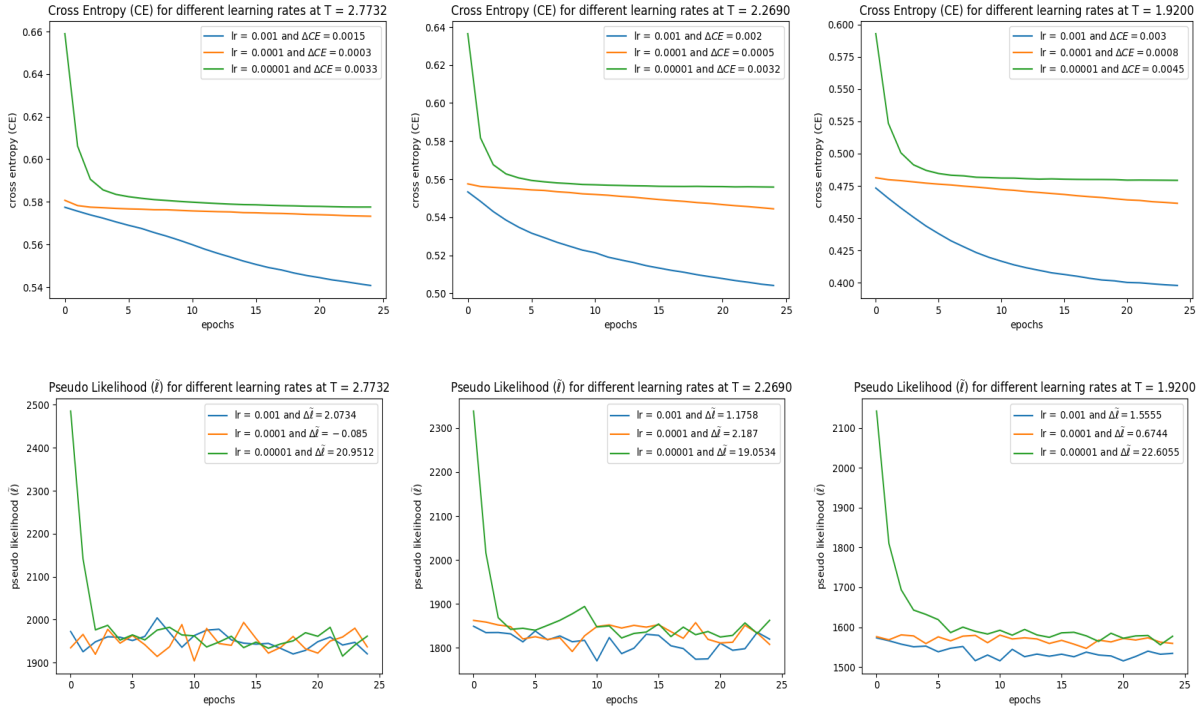

Figure 12: Graphs showing effect that learning rates ($\{0.001, 0.0001, 0.00001\}$) have on the training of the RBM over 20 epochs. (Top) Cross Entropy over the epochs, (Buttom) Pseudo Log-Likelihood over epochs.

First focusing on the CE, the graphs depicting the CE over the various temperatures seem to all follow a similar trend. The $\gamma = 0.0001$ drops vary fast and then plateaus, $\gamma = 0.001$ decreases but vary steadily, while $\gamma = 0.001$ seems to decrease the best, and at low temperatures starts to plateau close to 25 epochs.

In regards to the smallest learning rate, it appears to perform fairly well, then it reaches a suboptimal local minima at which point the parameters get stuck. Because the model cannot escape the local minima, due to its small step size, the weights can no longer be updated which leads to a plateau in CE. $\gamma = 0.0001$ is observed to improve consistently, but because of the small step size, there are not dramatic improvements after each epoch. Where $\gamma = 0.001$ appear to have the best overall performance, since the step sizes are most likely large enough to traverse the energy landscape efficiently without getting caught in any suboptimal local minima.

The rate at which the $\gamma = 0.0001$ model decrease in CE increases as temperature decrease. This is due to the effect that T has on the energy landscape that the RBM aims to learn. At high temperatures, the spins are randomly orientated which means that multiple different lattice configurations appear more or less similar, thus creating a jagged energy landscape. Because of the small learning rate of 0.0001, it is easy for them model to get trapped in certain places in the landscape. At $T \approx T_c$ the cluster islands form, making the distance between observations farther apart in the energy landscape, thus the energy landscape is more smooth, allowing the RBM to traverse it more easily. At low T, the observations are still very close to each other, but the spins are not orientated randomly. Therefore, RBM can still learns how to better reconstruct the observations.

When considering $\widetilde{\ell}$, the various learning rates seem to all stay more or less the same over the epochs, with $\gamma = 0.001$ being on average lower then the other two learning rates across the different temperatures.

Therefore, since $\gamma = 0.001$ produces the best decrease in CE and has an overall lower $\widetilde{\ell}$, subsequent models will be trained with a learning rate of 0.001.

The Contrastive Divergence Algorithm

The CD-k algorithm begins by sampling an initial visible state $\mathbf{v}_{data}$ from the training data, and fix the initial visible state $\mathbf{v}^{(0)}$ as this sample.

1. **Positive Phase**

   With $\mathbf{v}^{(0)}$ fixed, one can compute the conditional probabilities of the hidden layer via:

   $$P(h_j = +1 \mid \mathbf{v}^{(0)})) = \sigma\left(\beta\left(\sum_i v_i^{(0)} w_{ij} + b_j\right)\right)$$

   After sampling these probabilities, a threshold is applied such that if the $P(h_j = +1 \mid \mathbf{v}^{(0)})) > 0.5$, $h_j = +1$. After sampling for all hidden units, $\mathbf{h}^{(0)}$ is obtained. The update terms can now be approximated as follows:

   $$\langle v_i h_j \rangle_{data} \approx v_i^{(0)} \cdot h_j^{(0)}$$

   For all combinations of hidden and visible units. Additionally:

   $$\langle v_i \rangle_{data} \approx v_i^{(0)}$$

   $$\langle h_j \rangle_{data} \approx h_j^{(0)}$$

2. **Negative Phase**

   The visible layer is initialised with the same sample selected from the training data $\mathbf{v}^{(0)}$. The algorithm starts by running k steps of Gibbs sampling as detailed in section 4.1.2 on this $\mathbf{v}^{(0)}$. hidden and visible units are consecutively sampled using the Gibbs sampling algorithm in order to obtain $\mathbf{v}^{(k)}$ and $\mathbf{h}^{(k)}$.

   One can now approximate the anti-Hebbian term as follows:

   $$\langle v_i h_j \rangle_{model} \approx v_i^{(k)} \cdot h_j^{(k)}$$

   For all combinations of hidden and visible units. Additionally:

   $$\langle v_i \rangle_{model} \approx v_i^{(k)}$$

   $$\langle h_j \rangle_{model} \approx h_j^{(k)}$$

3. For each iteration the algorithm is run, a new $\mathbf{v}^{(0)}$ is sampled from the training data each time, and the positive and negative phase are both run in order to update the weight and bias parameters.

CD-k is so effective, as although the gradients it calculates are only approximations, they are in practice accurate enough using as few as $k = 1$ Gibbs sampling steps. As a result, the model can learn extremely efficiently the patterns in the training data by sequentially updating it's weights and biases.

4.1.4 Applying Persistent Contrastive Divergence (PCD-k)

The PCD-k learning algorithm is a variant of the CD-k algorithm discussed prior, and their differences lie in how they approach the negative sampling phase. In the CD-k algorithm, the negative sampling phase started by sampling a new training observation $\mathbf{v}^{(0)}$ at each iteration. Because the Gibbs sampling chain is initialised by a data observation, the chain starts off very close to the data distribution, allowing for a faster convergence. While this is efficient, it often leads to bias as the chain is not sufficiently allowed to explore the model distribution before computing the weight updates.

PCD-k aims to account for this bias, by maintaining the same visible state sampled from previous iterations of the algorithm. So the updated algorithm for the negative phase is:

1. Sample an initial $\mathbf{v}^{(0)}$ that is a random configuration of spins.
2. Perform k steps of the Gibbs sampling algorithm in order to acquire $\mathbf{v}^{(k)}$ and $\mathbf{h}^{(k)}$.
3. Calculate the weight updates as before in CD-k.
4. Return to step 1, however instead of randomly sampling a new state, use $\mathbf{v}^{(k)}$ as the initial state for the next iteration of the algorithm.

Comparing CD-k and PCD-k

In CD-k, the negative phase is initialised by a training data example for the visible layer. This means the Gibbs sampling chain starts very close to the training data's distribution, and may not be able to sufficiently explore the model's current learned distribution.

In contrast, PCD-k initialises each subsequent negative phase's visible layer using $\mathbf{v}^{(k)}$ computed in the previous iteration of the algorithm after k Gibbs sampling steps. The result of this is that the data generated by the Gibbs sampler is a much more unbiased representation of the model's distribution, as the persistent chain is able to explore the model's distribution over time instead of being reset at each iteration.

4.1.5 Applying Momentum

Momentum is a popular machine learning technique derived from classical physics, and is used on gradient optimisation problems typically where the cost function is complex and has many local minima and saddle points. Momentum is applied to the contrastive divergence training algorithm and helps prevent the algorithm from getting stuck in suboptimal regions by maintaining a velocity based on the magnitude of the previous update which creates an inertia effect preventing premature convergence.

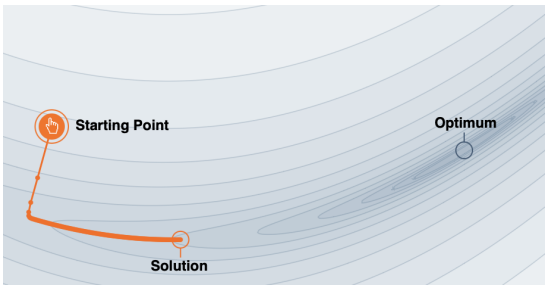

Figure 13: Gradient based optimisation done without momentum [6]

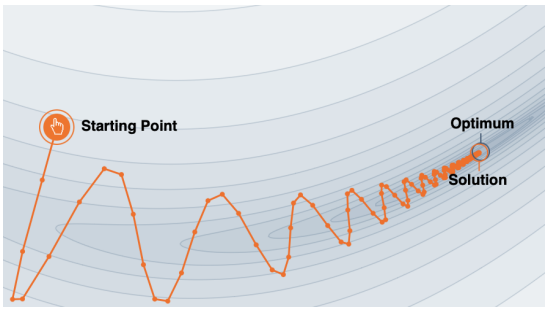

Figure 14: Gradient based optimisation done with momentum [6]

Momentum Algorithm

1. **Initialize velocity terms**:
$$\Delta W = 0, \quad \Delta a = 0, \quad \Delta b = 0$$
2. **Apply contrastive divergence:**
Using Contrastive divergence calculate the estimates for:
$\langle v_i h_j \rangle_{data}$, $\langle v_i h_j \rangle_{model}$, $\langle v_i \rangle_{data}$, $\langle v_i \rangle_{model}$, $\langle h_j \rangle_{data}$, $\langle h_j \rangle_{model}$
3. **Update velocity terms with momentum**:
$$\Delta W = \gamma \left(\langle v_i h_j \rangle_{\text{data}} - \langle v_i h_j \rangle_{\text{model}} \right) + \alpha \Delta W$$
$$\Delta a = \gamma \left(\langle v_i \rangle_{\text{data}} - \langle v_i \rangle_{\text{model}} \right) + \alpha \Delta a$$

$$\Delta b = \gamma \left(\langle h_j \rangle_{\text{data}} - \langle h_j \rangle_{\text{model}} \right) + \alpha \Delta b$$

Where α is the momentum coefficient, and determines how quickly the contributions of the previous gradients decay. The larger α is, the greater the impact previous gradients have.

4. **Update parameters**:

$$W = W + \Delta W$$

$$a = a + \Delta a$$

$$b = b + \Delta b$$

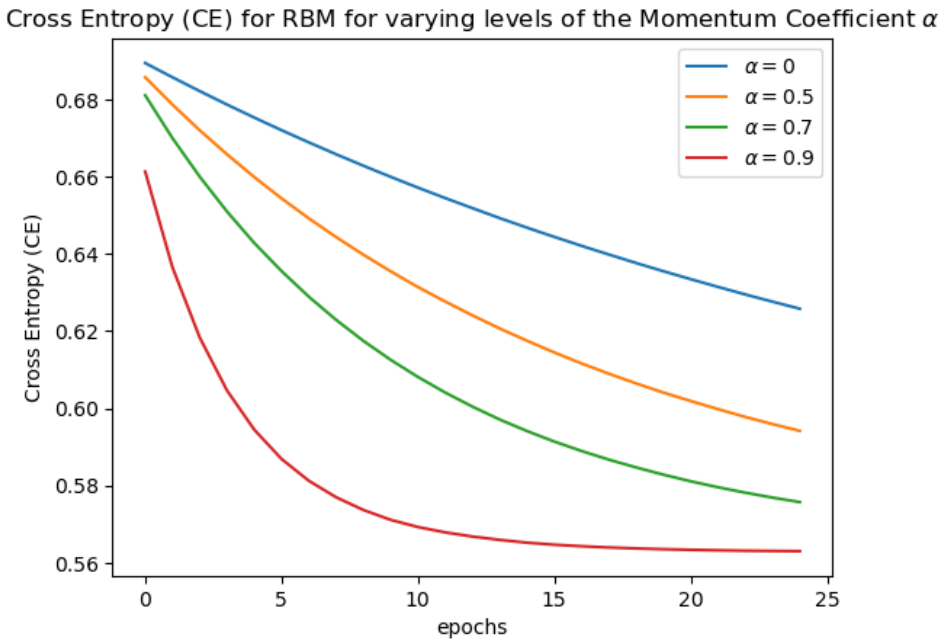

Figure 15: Empirically measurements of cross entropy during training for varying values of the momentum coefficient α for an RBM for $T = T_c$ with size $L = 64$

One can clearly observe in Figure 10 that by increasing the value of α, the convergence towards the optimum set of parameters is accelerated. Because of this, a final momentum coefficient value of $\alpha = 0.9$ is selected for the final RBM model.

A brief evaluation can be made between PCD, CDk and CDk with momentum $= 0.9$. Different RBM are trained, each using one of the different learning algorithms. Resulting in Figure 16. As can be seen from the figure, PCD appears to offer no clear improvement over CDk, whilst CDk with momentum performs the best. Therefore, the final model will be utilizing the CDk algorithm with momentum applied to the learning rate.

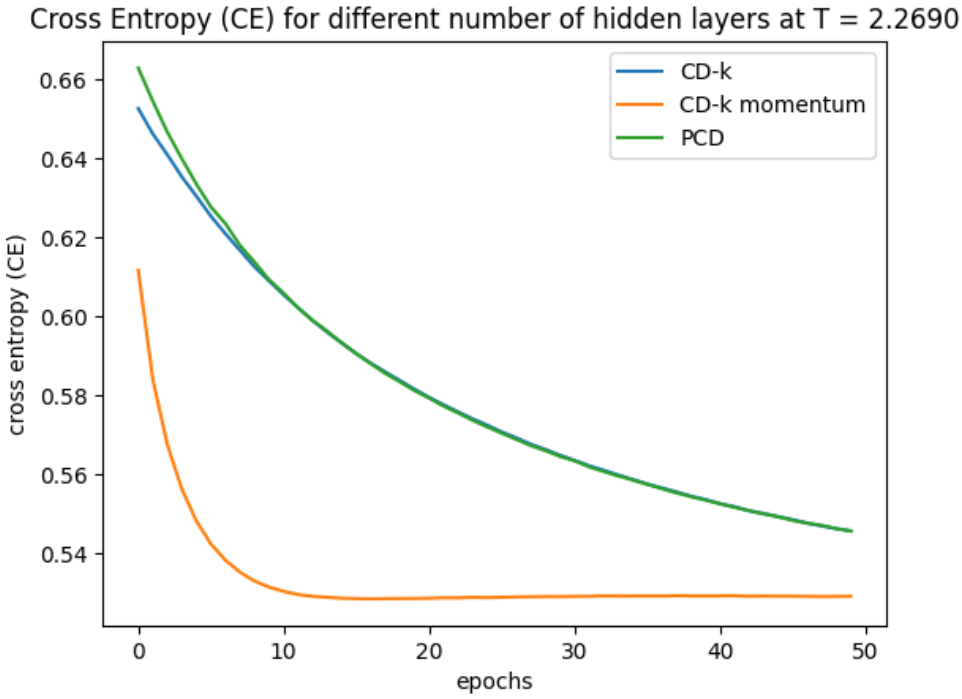

Figure 16: Graph showing the difference in CE for PCD, CDk and CDk with momentum = 0.9

4.2 Final RBM Models

4.2.1 Final Model Structure

For the final model building, due to time and computational constraints 3 temperature models are considered, one below, above, and at the critical temperature T_c. These temperatures are $T = 1.92,\ 2.26,\ 3.12$, and should offer insight into how the RBM model performs at and around the critical temperature.

These 3 models are trained with the following parameters:

1. $\mathbf{h} = 900$
2. $k = 1$
3. $\gamma = 0.001$
4. batch size = 128
5. epochs = 40

4.2.2 Comparing the MCMC generated vs. RBM generated Ising configurations

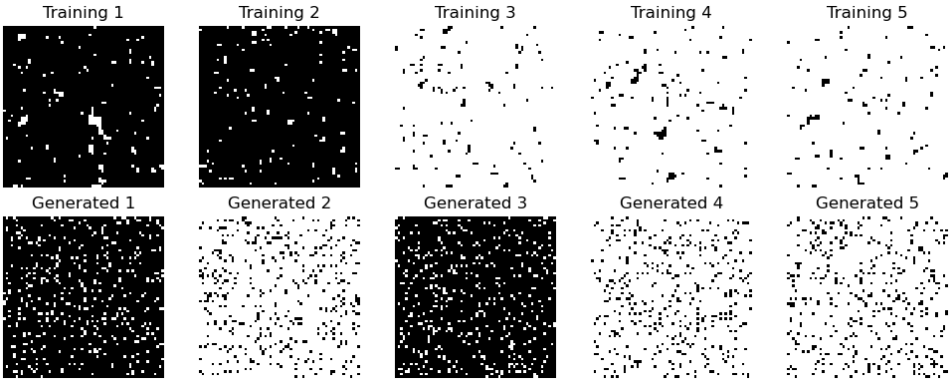

Figure 17: Comparison of the MCMC generated Ising configurations at $T = 1.92$ after 40 epochs

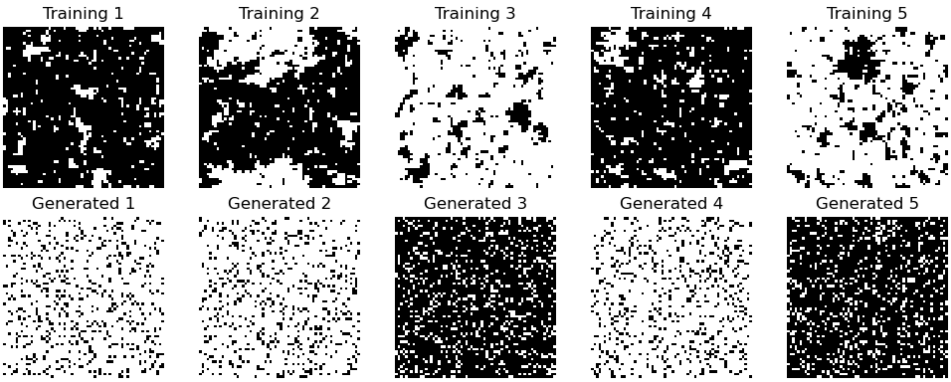

Figure 18: Comparison of the MCMC generated Ising configurations at $T = 2.26 = T_c$ after 40 epochs

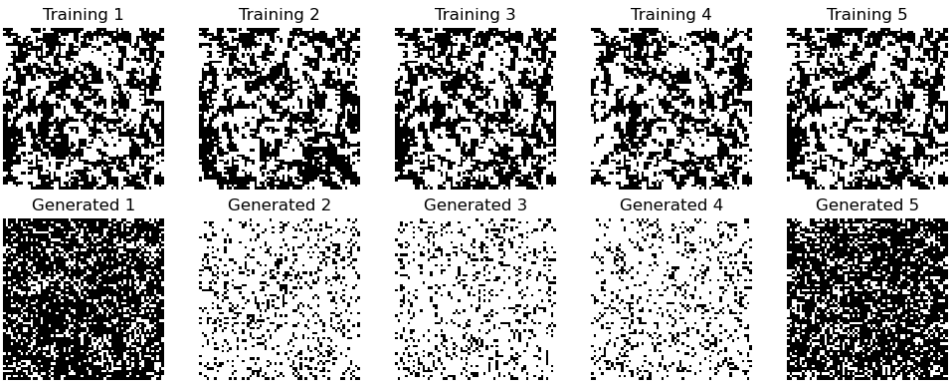

Figure 19: Comparison of the MCMC generated Ising configurations at $T = 3.12$ after 40 epochs

In the above comparisons in which the top row are generated by the MCMC simulation using the Wolff algorithm, it can be observed that over 40 epochs the RBM is able to gain a general understanding of the data it was trained on. However, the models clearly lack a nuanced understanding of spin correlations and cluster formations, as one can observe the RBM struggles especially near criticality, to simulate the same cluster dynamics that can be observed in the original data. The reason for this is likely that the models did not have sufficient time in epochs to train, as during the simulation the cross entropy was observed to be steadily decreasing, however the simulations were extremely computationally intensive without the access to industrial grade Graphical Processing Units (GPU's). In future, it would be interesting to

explore simulating these models with more epochs, as is currently indicated in the literature that many require in the thousands of epochs to become sufficiently capable at modelling the data near criticality. [7]

Comparing the distribution of Absolute Magnetisation $|M|$ per spin

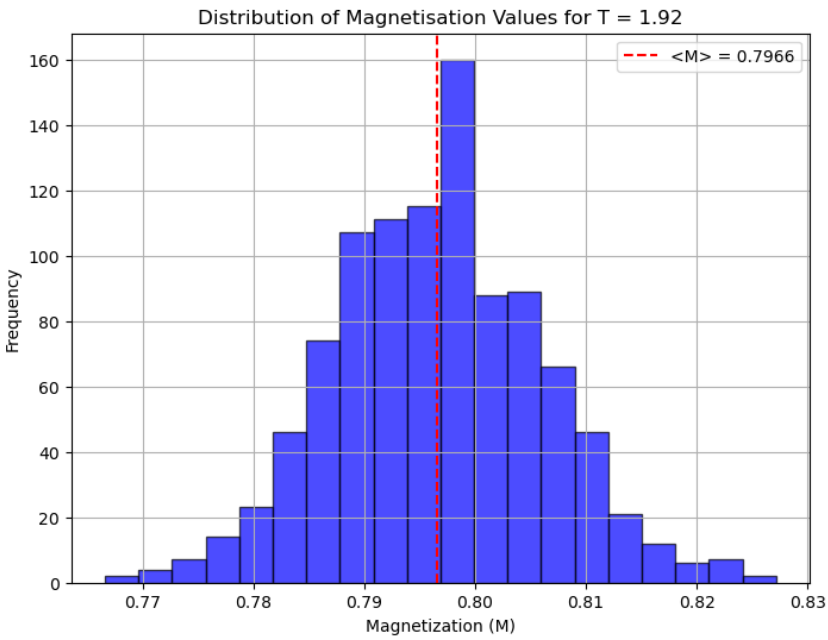

Figure 20: An empirical distribution of the absolute Magnetisation $|M|$ per spin for the states generated by the RBM for $T = 1.92$

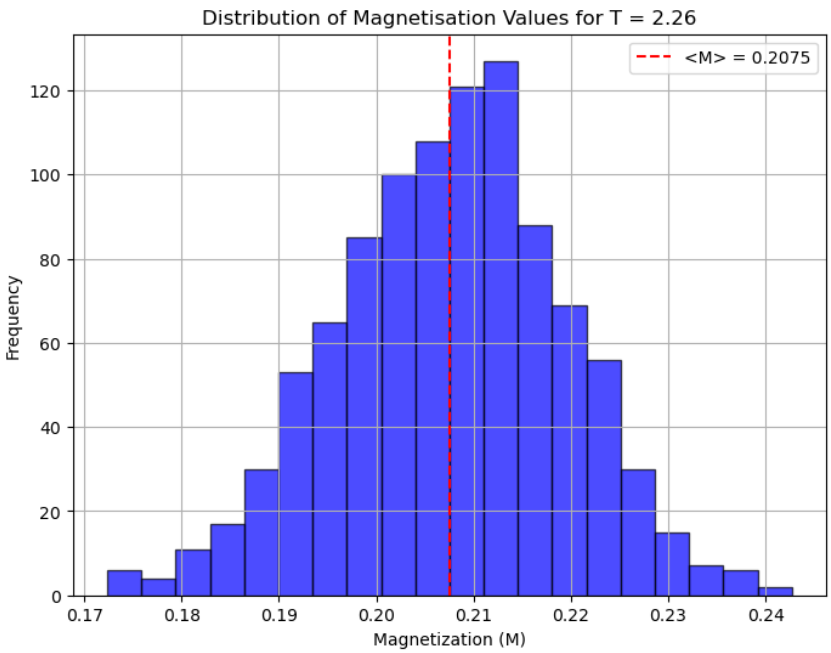

Figure 21: An empirical distribution of the absolute Magnetisation $|M|$ per spin for the states generated by the RBM for $T = 2.26$

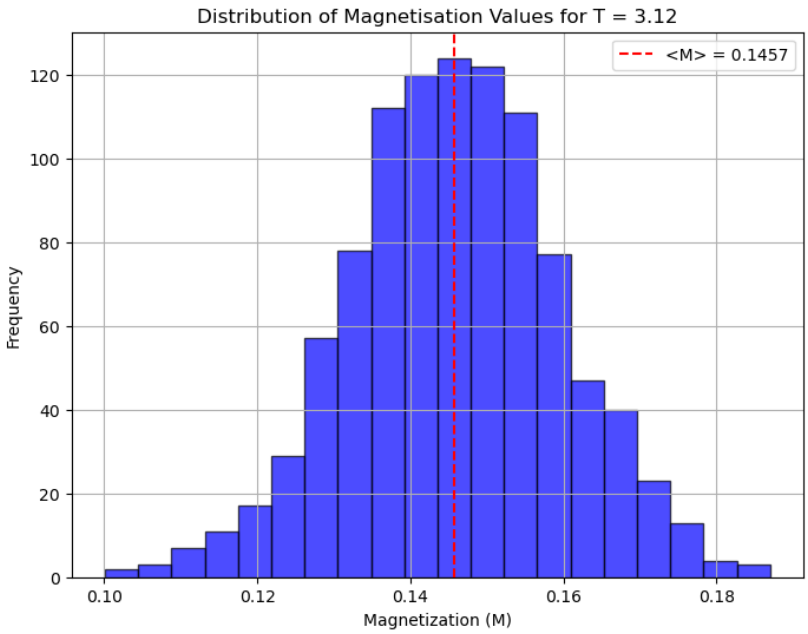

Figure 22: An empirical distribution of the absolute Magnetisation $|M|$ per spin for the states generated by the RBM for $T = 3.12$

All distributions for $|M|$ for temperatures $T = 1.92, 2.26, 3.12$ appear to be roughly normally distributed, centered around their sample means which estimate $\langle |M| \rangle$ as 0.7966, 0.2075, and 0.1457 respectfully. This indicates that the RBM has learned that the lower temperature states on average have a higher average magnetisation per spin, which is additionally substantiated by the generated states in Figures 15, 16 and 17 displaying varying levels of aligned spins (spins being more aligned for the temperature below T_c, and more random above T_c.

However, a clear bias can be observed in the results, as one would expect the distribution for $T = 2.26$ $T = 3.12$ to be have an expected value of $\langle |M| \rangle = 0$, however they both have the respective values: $\langle |M| \rangle = 0.2075$ and $\langle |M| \rangle = 0.1457$, which indicated the RBM's have not fully learned the distributions in these temperature states.

4.3 Conclusion

In conclusion, the RBM is an extremely complex and intricate model that is able to capture extremely nuanced patterns in certain data. However, when practically implemented it tends to suffer several drawbacks. Firstly, due to it's complex nature it tends to require a lot of computational power, as it's constantly updating and fine tuning an extraordinary amount of parameters. Additionally, the model is extremely dependent on the data it has been trained on, and any statistical error recorded in the training data has the potential to be learned by the model, leading to overfitting (the RBM modelling statistical error).

There are many possible areas for improvement for this investigation, one of the primary ones being more time spent training. In an ideal world, one would train the model for as long as needed, and stop training as soon as the validation error stops decreasing. Additionally, one could investigate using a more detailed dataset, with more observations of lattice configurations, and possibly more lattice units in order to decrease finite size effects. Additionally, there are a plethora of machine learning methods which are constantly evolving and improving in order to increase the robustness of models. Methods such as dropout, weight penalisation, and early stopping are all methods that help models to not overfit the data, which could all be applied to the RBM.

References

[1] Hopfield networks is all you need, (2020).

[2] Crouse E. Hopfield networks: Neural memory machines. *Towards Data Science*, (2022).

[3] D.P. Landau and K. Binder. *A guide to Monte Carlo simulations in Statistical Physics*. Cambridge University Press, (2000).

[4] Lars Onsager. Crystal statistics. i. a two-dimensional model with an order-disorder transition. *Phys. Rev.*, 65:117–149, Feb 1944.

[5] Goodfellow I. Bengio Y. and Courville A. *Deep Learning*. MIT Press, (2016).

[6] Alexander Alemi, David Balduzzi, Ian Fischer, and Marc Lanctot. Visualizing and understanding momentum. *Distill*, April 2017. Creative Commons Attribution license.

[7] Xiaobei (Emma) Zhang. Probing the ising model's thermodynamics through restricted boltzmann machines. 2024. HMC Senior Theses. 280.

[8] Jing Gu and Kai Zhang. Thermodynamics of the ising model encoded in restricted boltzmann machines. *Entropy*, 24(12):1701, November 2022.

[9] Bryan Clark. Restricted boltzmann machines — computing in physics. 2022.

[10] Timothy Budd. Criticality & cluster algorithms, 2024. Accessed: 2024-11-15.

[11] Fischer A. and Igel C. Empirical analysis of the divergence of gibbs sampling based learning algorithms for restricted boltzmann machines. *Lecture notes in computer science*, page 208–217, 2010.

[12] Qwertyus. Restricted boltzmann machine.svg, 2012. Accessed: November 15, 2024.